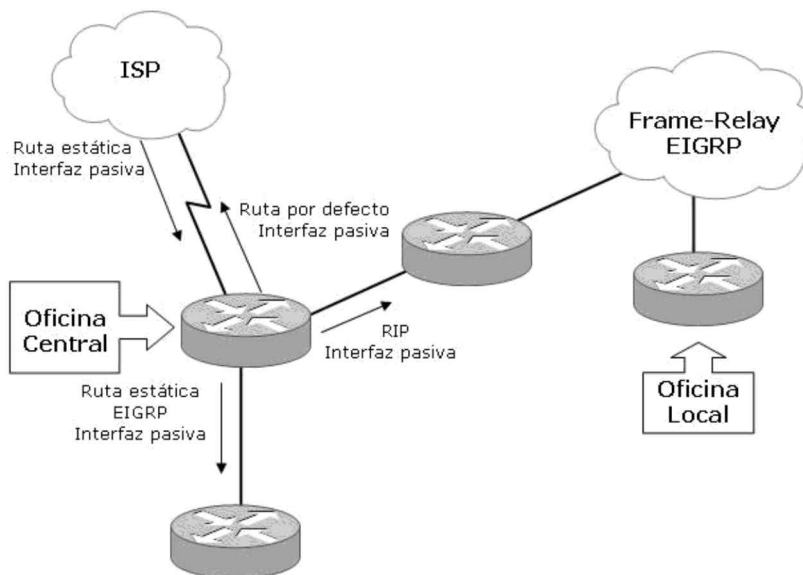


EIGRP se está ejecutando en la nube Frame-Relay evitando sobrecargas en la red, mientras que en los segmentos LAN no hay necesidad de protocolo de enrutamiento. RIP es el protocolo configurado en la oficina principal.

Los servidores contenidos en la oficina principal necesitan tener conocimiento de las redes EIGRP, por lo tanto habrá que redistribuirlas en RIP. El flujo inverso no es necesario evitando así el tráfico bilateral y consumo de recursos.



La configuración es sencilla porque no hay enlaces redundantes. La nube Frame-Relay utiliza enlaces punto a punto, en un futuro se podrían añadir más enlaces a la nube de manera redundante.

Este ejemplo es una solución simple que le permite a la organización el uso del hardware existente.

Ejemplo 2. Redistribución de rutas con caminos redundantes. Este caso cubre la redistribución con caminos redundantes entre diferentes protocolos de enrutamiento y resuelve los problemas de selección de rutas que resultan con las redes redistribuidas.

En la topología de la figura el router A está conectado a las redes 120.100.1.0, 120.100.2.0 y 120.100.3.0. Utilizando RIP la red 120.100.1.0 es advertida al router B, la red 120.100.3.0 al router C y la red 120.100.2.0 es advertida al router B y al router C.

La siguiente es la información contenida en la tabla de enrutamiento del router B:

Protocolo de enrutamiento	Red/subred	Próximo salto	Métrica
RIP	120.100.1.0/24	120.100.3.1	1 salto
RIP	120.100.2.0/24	120.100.3.1	1 salto
Conectado	120.100.3.0/24	Conectado E0	0
Conectado	10.10.10.8/30	Conectado S0	0
Conectado	10.10.10.12/30	Conectado S0	0
Conectado	10.10.10.16/30	Conectado S0	0
EIGRP	10.10.10.20/30	10.10.10.9	4201056
EIGRP	10.10.10.24/30	10.10.10.9	4201056
EIGRP	10.10.10.28/30	10.10.10.13	4201056
EIGRP	10.10.10.32/30	10.10.10.13	4201056

La tabla de enrutamiento del router A contiene todas las subredes de la red 10.0.0.0 255.255.255.252. Debido a que RIPv1 no envía la máscara de red en las actualizaciones y el router A no está conectado a dicha red se deberá configurar RIPv2 entre los router A y router B.

La tabla de enrutamiento ve a todos los caminos como únicos, por lo que es necesario saber cuáles son las rutas accesibles por medio de RIP o EIGRP. La tabla de enrutamiento se mantiene igual aún después de la redistribución.

Dependiendo de los tiempos de las actualizaciones y de la velocidad de convergencia, el router C podría estar desactualizado debido a que los router E, router F y router G le envían información sobre cómo llegar a las redes 120.100.2.0 y 120.100.3.0.

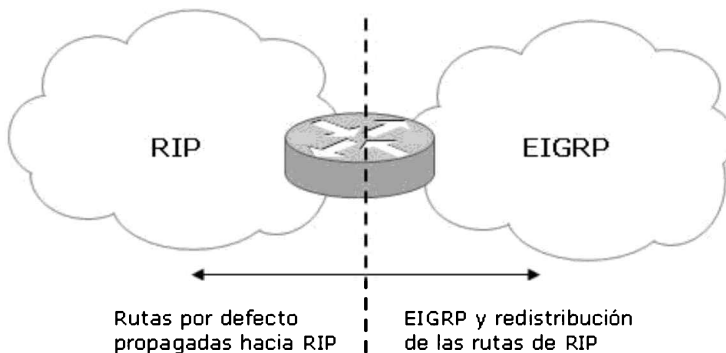
A su vez el router C también recibe información del router A. Evidentemente enviar el tráfico hacia esas redes por el router A es la mejor opción, pero como EIGRP tiene una distancia administrativa menor, la ruta para EIGRP es colocada en la tabla de enrutamiento. Suponiendo que la nube Frame-Relay posea caminos iguales para todos los enlaces, las tablas de enrutamiento distribuyen el tráfico equitativamente por las tres rutas.

Las siguientes sintaxis muestran como las configuraciones favorecen a la distancia administrativa de RIP para las redes LAN dentro de su dominio. Los caminos de RIP serán elegidos, las redes 120.100.1.0 y 120.100.2.0 tendrán una distancia administrativa de 200 con la ayuda de listas de acceso.

```
Router(config)# router rip
Router(config-router)# network 120.100.0.0
Router(config-router)# passive interface S0.1
Router(config-router)# redistribute eigrp 100 metric 3
Router(config)# router eigrp 100
Router(config-router)# network 120.100.0.0
Router(config-router)# passive interface E0
Router(config-router)# default-metric 10000 100 255 1 1500
Router(config-router)# distance 200 0.0.0.0 255.255.255.255 3
Router(config)# access-list 3 permit 120.100.1.0
Router(config)# access-list 3 permit 120.100.2.0
```

El comando **distance** configura la distancia administrativa para el proceso EIGRP 100 de 90 a 200 haciendo que las rutas de RIP sean las óptimas debido a que la distancia administrativa de RIP es 120. Añadiendo la granularidad de poder elegir las redes que necesitan ser utilizadas en lugar de todas con el uso de las ACL. De esta forma las redes 120.100.1.0 y 120.100.2.0 serán afectadas por el comando **distance**. El uso de la red 0.0.0.0 con la máscara 255.255.255.255 es un comodín.

Ejemplo 3. Redes por defecto en un entorno de redistribución. La utilización de redes por defecto simplifica la configuración de la distribución permitiendo redistribuciones en un sólo sentido o camino. Esto reduce significativamente la posibilidad de bucles hacia el dominio origen.



En este diseño cada router dentro del dominio de RIP tiene conocimiento de todas las redes internas pero las otras redes son accesibles vía la red por defecto. La configuración del router B se muestra en la siguiente sintaxis:

```
RouterB(config)# router rip
RouterB(config-router)# network 190.10.10.0
```

El router A redistribuye entre RIP y EIGRP con el dominio de RIP actuando como una red stub, la ruta por defecto es configurada como una ruta estática en el router A y redistribuida dentro de RIP para propagarse en todo su dominio.

Los routers internos que sólo ejecutan RIP deben estar preparados para recibir la ruta por defecto, la red de destino sólo será alcanzable por medio de esta ruta por defecto.

La siguiente es la sintaxis de la configuración del router A:

```
RouterA(config)# router eigrp 100
RouterA(config-router)# redistribute rip
RouterA(config-router)# default-metric 10000 100 255 1 1500
RouterA(config-router)# network 10.0.0.0
RouterA(config)# router rip
RouterA(config-router)# network 190.10.10.0
RouterA(config-router)# default-metric 3
RouterA(config-router)# redistribute static
RouterA(config)# ip route 0.0.0.0 0.0.0.0 10.10.10.1
```

La redistribución en router A es sólo hacia un sentido, EIGRP necesita conocer todas las redes en el dominio de RIP pero en cambio RIP no necesita conocer las redes del mundo exterior.

CONTROL DE LAS ACTUALIZACIONES DE ENRUTAMIENTO CON FILTRADO

Algunos mecanismos, para controlar las actualizaciones de enrutamiento, se han visto en los párrafos anteriores, pero a veces es necesario utilizar herramientas más potentes y flexibles. Estos mecanismos son listas de acceso que aplicadas a las actualizaciones de enrutamiento se denominan listas de distribución (*distribute lists*).

El proceso y la lógica son similares a las ACL:

1. El router recibe una actualización de enrutamiento o está a punto de enviar actualizaciones sobre una o más redes.
2. El router controla si la interfaz implicada en la acción tiene aplicados filtros.
3. Si existe un filtro, el router examina la ACL para ver si existen coincidencias en alguna de las redes en la actualización de enrutamiento.
4. Si no hay coincidencias en los filtros o no existen, la red se envía al proceso de enrutamiento como ocurre normalmente.
5. Si hay coincidencia en un filtro la ruta es procesada acorde a la sentencia de la lista de distribución. Dicha ruta se anunciara si coincide con un *permit* o se descartará si coincide con un *deny*.
6. Si no se encuentra ninguna coincidencia al final de la lista de distribución, una denegación implícita descartará la actualización.

Las actualizaciones de enrutamiento pueden ser filtradas por protocolos de enrutamiento, definiendo una ACL y aplicándola al protocolo. Existen ciertas limitaciones en las listas de distribución cuando son aplicadas a OSPF, las configuradas como entrantes previenen que las rutas se añadan en la tabla de enrutamiento, pero no previenen que las LSA se propaguen. Por este motivo las listas de distribución son empleadas prudentemente en los ABR o ASBR.

Al crear una lista de distribución se siguen los siguientes pasos:

1. Identificar la dirección de red que ha de ser filtrada y crear la lista de acceso. Permitir las redes que deben ser anunciadas.
2. Determinar si el filtrado es entrante o propagadas hacia otros routers o salientes.
3. Asignar la ACL utilizando el comando **distribute-list**.

La sintaxis completa del comando en la forma entrante es la siguiente:

```
Router(config-router)# distribute-list {access-list-number / name}  
in [type number]
```

Comando	Descripción
<i>access-list-number / name</i>	Especifica una ACL numerada o nombrada.
in	Aplica la ACL como entrante.
type number	Opcional, especifica información sobre la interfaz.

La sintaxis completa del comando en la forma saliente es la siguiente:

```
Router(config-router)# distribute-list {access-list-number / name}
out [interface-name | routing-process | autonomous-system-number]
```

Comando	Descripción
<i>access-list-number / name</i>	Especifica una ACL numerada o nombrada.
out	Aplica la ACL como saliente.
<i>interface-name</i>	Opcional, nombre de la interfaz donde se producirá el filtrado.
<i>routing-process</i>	Opcional, especifica el nombre del proceso de enrutamiento, o si es estático o directamente conectado.
<i>autonomous-system-number</i>	Opcional, número del sistema autónomo del proceso de enrutamiento.

VERIFICACIÓN, MANTENIMIENTO Y RESOLUCIÓN DE FALLOS

Para que la redistribución funcione adecuadamente y simplificar al máximo los problemas que puedan surgir es necesario tener un conocimiento preciso de la topología tanto física como lógica. Conocer el volumen y los picos de tráfico también es un tema importante para entender los conflictos de conectividad dentro de la red.

Los siguientes comandos ayudarán a entender y resolver problemas ocasionados en la redistribución:

- **show ip protocol**
- **show ip route**
- **show ip route *routing-protocol***
- **show ip eigrp neighbors**
- **show ip ospf database**

El comando traceroute: este comando puede ser empleado desde el modo usuario, pero se necesita la versión extendida y debe emplearse desde el modo privilegiado. Mostrará la información de todos los routers que un paquete ha pasado desde que ha sido enviado. A la versión extendida del comando **traceroute** se accede pulsando un intro; diferentes cuestiones son presentadas que permitirán la modificación de ciertos parámetros que vienen por defecto.

El comando extendido ping: para hacer comprobaciones básicas se puede utilizar el comando **ping** desde el modo usuario. Pero si se requiere emplear todo el potencial que brinda este comando se debe ejecutar desde el modo privilegiado, de esta forma se pueden modificar los parámetros por defecto y personalizarlo según se necesite.

Los comandos traceroute y ping extendido: el comando **traceroute** muestra el camino tomado y podrá en ciertos casos identificar el problema de la red. Si el comando **traceroute** falla indica que existe un problema y es un buen punto de partida para solucionarlo. El comando **ping** es muy útil porque puede anunciar cada una de las interfaces que va atravesando si la opción “record” es elegida. Aunque tiene una limitación de 9 saltos. Es posible utilizar una dirección IP origen en los comandos **traceroute** y **ping** extendidos, que debe ser una interfaz en el router de origen. Esto permitirá chequear ACL, mapas de ruta, etc. Cambiando las direcciones de origen desde todas las interfaces del router se obtiene una visión total del funcionamiento de la red.

CONTROL DE LA REDISTRIBUCIÓN CON ROUTE-MAPS

Funcionamiento de los route-maps

Los *route-maps* son herramientas que establecen coincidencias con un determinado patrón y que pueden utilizarse para cambiar la cabecera del paquete, el mapa de ruta, etc. Los route-maps son similares a las listas de acceso pero más complejos debido a que permiten hacer muchas más cosas.

Existen diferentes situaciones donde se pueden emplear los route-maps:

- **Controlar la distribución:** los route-maps pueden permitir o denegar rutas como así también cambiar métricas.
- **Definir políticas para PBR (Policy-Based Routing):** las políticas basadas en enrutamiento son creadas a partir de los route-maps para permitir al router tomar decisiones complejas basadas en diferentes criterios del enrutamiento tradicional.
- **Agregar granularidad a la configuración de NAT (Network Address Translation):** un escenario de NAT tradicional está limitado en la simple traducción de un direccionamiento estáticamente, con los route-maps pueden hacerse traducciones de manera arbitraria.
- **Implementar BGP PBR:** más adelante se detallará el funcionamiento en el capítulo de BGP.

Características de los route-maps

Los route-maps y las listas de acceso son similares en la lógica de funcionamiento, se procesan de arriba hacia abajo y en la primera coincidencia se ejecuta la acción permitiendo o denegando, aunque los route-maps permiten hacer muchas más cosas. Los route-maps pueden cambiar la dirección del próximo salto o la métrica de la cabecera IP. La cantidad de atributos que los route-maps pueden modificar es enorme. Las características de los route-maps se pueden resumir en los siguientes conceptos:

- Una colección de sentencias con el mismo nombre se considera un route-map al igual que ocurre con las ACL.
- Los route-maps son procesados se arriba hacia abajo hasta que se encuentra la primera coincidencia.

- Cada sentencia de los route-maps puede tener ninguna o más condiciones de coincidencia. Una sentencia que no tenga ninguna coincidencia se aplica a todo el tráfico, similar al *any* de las ACL. El tráfico que no encuentra coincidencia pasa a la siguiente sentencia. Existen dos cuestiones a tener en cuenta:
 - Una simple coincidencia puede contener múltiples condiciones. Si una de esas condiciones es verdadera, se considerará coincidencia. Esto es una operación lógica OR.
 - Si existen múltiples coincidencias todas ellas deben ser verdaderas para que la coincidencia sea efectiva. Esto se considera una operación lógica AND.
- Los route-maps tienen ninguna o más sentencias **set**, que definen la acción a tomarse. Los parámetros definidos en la sentencia **set** se aplican siempre que previamente haya habido una coincidencia.
- Cada sentencia de los route-maps tiene que permitir o denegar. Un tráfico permitido es afectado, mientras que un tráfico que no encuentra coincidencia no lo será.
- El tráfico que no es explícitamente permitido será implícitamente denegado.
- Dentro de un route-map cada sentencia tiene un número de secuencia y puede ser editada individualmente.
- El uso de los route-maps es muy común en redes que tengan redistribución. Pero la utilización de éstos debe hacerse con total conocimiento del funcionamiento de la red debido a que puede influenciarse incorrectamente el flujo de datos o crear bucles.

El siguiente ejemplo describe un route-map llamado **RMSIMPLE**:

```
access-list 23 permit 23.0.0.0
access-list 33 permit 33.0.0.0
access-list 43 permit 43.0.0.0
access-list 53 permit 53.0.0.0
!
route-map RMSIMPLE permit 10
  match ip address 23
  match interface FastEthernet0/0
  set ip next-hop 67.43.6.7
!
route-map RMSIMPLE permit 20
  match ip address 33
!
route-map RMSIMPLE permit 30
```

```
match ip address 43 53
set ip next-hop 213.35.32.78
!
route-map RMSIMPLE permit 40
set ip next-hop 76.45.23.7
```

Todos los route-maps con el mismo nombre forman parte de una lista.

1. Los route-maps están organizados por número de secuencia. En este ejemplo es 10, 20 o 30.
2. Cada sentencia puede tener ninguna o más coincidencias:
 - En la sentencia 10 hay dos condiciones de coincidencia. Ambas deben ser verdaderas para que el **set** se aplique.
 - En la sentencia 20 sólo hay una coincidencia, de manera que los paquetes denegados no serán considerados. En caso de que este route-map se aplicara a la redistribución, las rutas denegadas no serían redistribuidas.
 - La sentencia 30 tiene dos coincidencias y con que sólo una fuera verdadera el **set** se aplicará.
 - La sentencia 40 no tiene ningún estado de coincidencia, lo que implica que se le aplicará a todo.
3. Cada sentencia de los route-maps puede tener ninguna o más sentencias **set**:
 - Las sentencias 10, 30 y 40 modifican el siguiente salto para todas las coincidencias.
 - La sentencia 20 no posee sentencias **set**, por lo tanto la única acción posible será denegar o permitir.
4. El tráfico que coincide con un *permit* es afectado por el route-map, mientras que el que coincide con un *deny* o no está en la lista no es afectado por el route-map.
 - La sentencia 20 previene que las coincidencias no sean afectadas por la lista, por lo tanto si hay alguna coincidencia está no se le aplicará un siguiente salto diferente.
 - Finalmente hay un *permit any* en la sentencia 40 por lo tanto todo el tráfico será afectado.

CONFIGURACIÓN DE LOS ROUTE-MAPS

La siguiente sintaxis muestra la configuración de los route-maps:

```
Router(config)# route-map map-tag [{permit | deny} sequence-number]
```

Para borrar por completo el route-map se deberá agregar un **no** inicial al comando anterior.

Comando	Descripción
<i>map-tag</i>	Nombre del route-map.
permit deny	Si existe una coincidencia en el criterio y existe un <i>permit</i> , el paquete es enviado tal como se haya definido en las acciones <i>set</i> , si no hubiera coincidencia pasará a la siguiente sentencia.
<i>sequence-number</i>	Indica el orden de procesamiento de las secuencias, si no se especifica los hará automáticamente de 10 en 10.

Uno de los potenciales de los route-maps es la posibilidad de cambiar la forma en que el router procesa las rutas y fundamentalmente la opción de cambiar la métrica.

Comandos match para la redistribución con route-maps

La siguiente tabla resume los comandos **match**. Estos comandos son usados para determinar si la ruta va a ser redistribuida.

Comando	Descripción
match interface (IP)	Distribuye rutas que tienen un próximo salto alcanzable desde la interfaz de salida que se ha especificado. Hace que coincida cualquier ruta que tenga su próximo salto por alguna de las interfaces que se están listando.
match ip address [access-list-number name]	Describe el número o nombre de la lista.

<code>match ip next-hop</code>	Identifica las rutas que poseen el próximo salto que se indica.
<code>match ip route-source</code>	Redistribuye las rutas que han sido advertidas desde la dirección especificada.
<code>match metric</code>	Redistribuye las rutas con la métrica especificada.
<code>match route-type (IP)</code>	Redistribuye las rutas del tipo especificado.
<code>match tag</code>	Redistribuye las rutas en la tabla de enrutamiento que coinciden con las etiquetas especificadas.

Comandos set para la redistribución con route-maps

Los comandos **set** funcionan a partir de una coincidencia previa. Mientras los comandos **match** determinan si la ruta va a ser redistribuida, el parámetro **set** determinará la forma en que va a ser redistribuida.

La sintaxis del comando **set** es la siguiente:

```
Router(config-route-map)# set {criteria}
```

La siguiente tabla resume los comandos **set** utilizados en la redistribución.

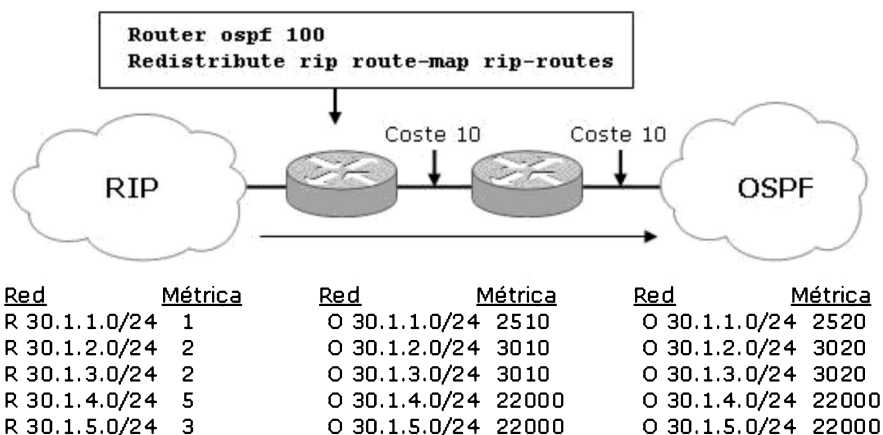
Comando	Descripción
<code>set level {level-1 level-2 level-1-2 stub-area backbone}</code>	Es utilizado por IS-IS para determinar el nivel del router que debería importar las rutas. OSPF lo utiliza para determinar el tipo de área/router desde el cual las rutas deberían ser importadas.
<code>set metric (BGP, OSPF, RIP)</code>	Configura la métrica para un determinado protocolo de enrutamiento.
<code>set metric-type {internal external type-1 type-2}</code>	Configura el tipo de métrica para el protocolo destino.
<code>set tag tag-value</code>	Configura un valor de etiqueta del protocolo de enrutamiento destino.

Una vez configurado, el route-map debe ser vinculado para que ejecute la acción. Este evento se describe en la siguiente sintaxis:

```
Router(config-router)# redistribute protocol [process-id] [route-map map-tag]
```

Donde **map-tag** es el nombre que se va a utilizar para la distribución y tiene que corresponderse con el nombre especificado con el comando **route-map**.

El ejemplo ilustra la funcionalidad del route-map, la sintaxis corresponde a la siguiente figura:



```
Router(config)# router ospf 100
Router(config-router)# redistribute rip route-map rip-routes
.....
Router(config)# route-map rip-routes permit 10
Router(config-route-map)# match metric 1
Router(config-route-map)# set metric 2500
Router(config-route-map)# set metric-type type-1
Router(config-route-map)# set tag 100
Router(config)# route-map rip-routes permit 20
Router(config-route-map)# match metric 2
Router(config-route-map)# set metric 3000
Router(config-route-map)# set metric-type type-1
Router(config-route-map)# set tag 200
Router(config)# route-map rip-routes permit 30

Router(config-route-map)# set metric 22000
```

Este route-map examina todas las actualizaciones desde RIPv2 y las redistribuye dentro de OSPF.

Las rutas con un número de saltos de 1 tendrán una métrica en OSPF con un valor de coste 2500, serán de tipo 1 y tendrán una etiqueta de 100. Las rutas con 2 saltos tendrán una métrica en OSPF con un valor de coste 3000, serán de tipo 1 y tendrán una etiqueta de 200. Todas las demás rutas tendrán un coste estático de 22000.

En cada una de las sentencias se modifica la métrica (coste) que las rutas tendrán una vez redistribuidas en OSPF. La configuración en type-1 indica que la métrica se incrementará a medida que los saltos aumenten; en cambio en type-2 (por defecto) la métrica permanecerá constante. Finalmente se etiquetan ciertas rutas con un valor de 100, otras con un valor de 200 y otras no han sido etiquetadas. Esto permitirá en otros puntos de redistribución hacer *match* en función de la etiqueta y permitir o denegar en base a esto.

Como nota al margen decir que las etiquetas (tags) son extremadamente útiles para escenarios con múltiples puntos de redistribución bidireccional.

VERIFICACIÓN DE LOS ROUTE-MAPS

La mayoría de los comandos para monitorizar el funcionamiento de los route-maps son los mismos empleados para verificar la redistribución:

- **show ip protocol**
- **show ip route**
- **show ip route *routing-protocol***
- **show ip eigrp neighbors**
- **show ip ospf database**
- **show route-map**

Tal como se explicó en la redistribución el comando **traceroute** y **ping** son de mucha utilidad.

INTRODUCCIÓN A DHCP

DHCP (Dynamic Host Control Protocol) descende del antiguo protocolo **BootP**, permite a un servidor asignar automáticamente a un host direcciones IPv4 y otros parámetros cuando está iniciándose.

DHCP ofrece dos principales ventajas:

1. DHCP permite que la administración de la red sea más fácil y versátil, de lo contrario habría que asignar manualmente el direccionamiento a todos los host, tarea bastante tediosa y que generalmente conlleva errores.
2. DHCP asigna direcciones IP de manera temporal creando un mayor aprovechamiento del espacio en el direccionamiento. Un ejemplo claro son los clientes DSL, ellos sólo necesitan la dirección IP cuando están conectados online.

El proceso DHCP sigue los siguientes pasos:

1. El cliente envía un broadcast preguntando por configuración IP a los servidores, los que responderán con una dirección IP, una máscara, una puerta de enlace y otras informaciones. En este paso el nuevo cliente ha enviado un mensaje en forma de broadcast llamado **DHCP discover**.

2. Cada servidor en la red responderá con un **Offer**. Comúnmente en una red pueden existir varios servidores DHCP de manera redundante, es normal que todos intenten responder. El mensaje offer contiene una dirección IP, una máscara, una puerta de enlace, el tiempo de alquiler y los posibles códigos de opción.
3. El cliente considera todas las ofertas y elige una. Esta opción dependerá del cliente. Por ejemplo Windows XP elegirá una dirección IP que previamente se le haya ofrecido, otros sistemas operativos simplemente aceptarán la primera oferta que les llegue. A partir de este momento el cliente envía un mensaje llamado **Request** en el que básicamente pregunta sobre la disponibilidad de la IP de la que tiene preferencia.
4. El servidor responde con un ACK informando a su vez que toma conocimiento que el cliente se queda con esa dirección IP.
5. Finalmente el cliente envía un ARP request para esa nueva dirección IP. Si alguien responde, el cliente sabrá que esa dirección está en uso y que ha sido asignada a otro cliente lo que iniciará el proceso DHCP nuevamente. Este paso se llama **Gratuitous ARP**.

Existen varios sistemas operativos clientes que utilizan **APIPA** (Automatic Private IP Addressing). Es un proceso por el cual el host configurado para un direccionamiento dinámico se asigna a sí mismo una dirección IP al no poder contactar con un servidor DHCP.

Las direcciones IP que se autoasignan son del rango 169.254.0.0. De la misma forma que en el quinto paso anterior se envía un **Gratuitous ARP** para verificar que nadie más posea dicha dirección antes de quedársela.

Finalmente, cuando se detecta un host con una dirección IP 169.254.X.X significa que no ha podido contactar con el servidor DHCP.

Dispositivos DHCP

Siempre habrá al menos dos dispositivos involucrados en el proceso de configuración del DHCP, un **Servidor DHCP** y un **Cliente DHCP**. Ocasionalmente habrá otros dispositivos opcionales llamados **DHCP relays**.

CONFIGURACIÓN DHCP

Configuración de un servidor DHCP

Los siguientes pasos describen la configuración de un router ejecutando IOS como servidor DHCP:

1. Crear un almacén (pool) de direcciones que serán asignadas a los clientes.

```
Router(config)# ip dhcp pool name
```

2. Determinar el direccionamiento de red y máscara que se le asignará al pool.

```
Router(config-dhcp)# network network/mask
```

3. Configurar el período que el cliente podrá disponer de esta dirección. Este período de alquiler suele ser de tres días; en las versiones de IOS es de un día, pero puede configurarse en horas, minutos y segundos.

```
Router(config-dhcp)# lease days
```

4. Identificar el servidor DNS.

```
Router(config-dhcp)# dns-server address
```

5. Identificar la puerta de enlace o gateway.

```
Router(config-dhcp)# default-router ip-address
```

6. Excluir si es necesario las direcciones que por seguridad o para evitar conflictos no se necesite que el DHCP otorgue.

```
Router(config)#ip dhcp excluded-address start-ip end-ip
```

Las direcciones IP son siempre asignadas en la misma interfaz que tiene una IP dentro de ese pool. La siguiente sintaxis muestra un ejemplo de configuración dentro de ese contexto:

```
Router(config)# interface fastethernet 0/0
Router(config-if)# ip address 192.168.1.1 255.255.255.0
Router(config)# ip dhcp pool 1
Router(config-dhcp)# network 192.168.1.0 /24
Router(config-dhcp)# default-router 192.168.1.1
Router(config-dhcp)# lease 3
Router(config-dhcp)# dns-server 192.168.77.100
```

Algunos dispositivos IOS reciben direccionamiento IP en algunas interfaces y asignan direcciones IP en otras. Para estos casos DHCP puede importar

las opciones y parámetros de una interfaz a otra. El siguiente comando para ejecutar esta acción es:

```
Router(config-dhcp)# import all
```

Este comando es muy útil cuando se debe configurar DHCP en oficinas remotas. El router una vez localizado en su sitio puede determinar el DNS y las opciones locales.

La siguiente tabla describe una serie de comandos opcionales:

Comando	Descripción
<code>service dhcp</code>	Habilita el servidor DHCP que por defecto ya lo está.
<code>ip dhcp database</code>	Configura un agente de base de datos.
<code>no ip dhcp conflict logging</code>	Deshabilita el login de conflictos.
<code>ip dhcp excluded-address start-ip end-ip</code>	Define las direcciones que serán excluidas del pool.
<code>domain-name domain</code>	DNS asignado por defecto.

Configuración de un DHCP Relay

Normalmente los routers no retransmiten tráfico broadcast. En algunos casos una excepción a esta regla podría ser muy útil. Un servidor DHCP centralizado que ofrezca un direccionamiento a todas las redes remotas sería más fácil de administrar y resolver posibles conflictos. Todos los routers en este caso deberían dejar pasar todo el tráfico broadcast originado por las transacciones DHCP.

Un router configurado para dejar pasar los **DHCP request** es llamado **DHCP Relay**. Cuando es configurado, el router permitirá el reenvío de broadcast que haya sido enviado a un puerto UDP determinado hacia una localización remota. El DHCP Relay reenvía los *requests* y configura la puerta de enlace en el router local.

```
Router(config-if)# ip helper-address address
```

Cuando se configura soporta 8 puertos UDP, cualquier broadcast que se reciba en estos puertos es reenviado a la dirección IP que se especifica en el comando. Los puertos son los siguientes:

- **NTP** (UDP 37)
- **TACACS** (UDP 49)
- **DNS** (UDP 53)
- **DHCP** (UDP 67 y 68)
- **TFTP** (UDP 69)
- **NetBIOS** name service (UDP 137)
- **NetBIOS** datagram service (UDP 138)

Adicionalmente se podrían agregar más puertos con el comando:

```
Router(config)# ip forward-protocol udp port
```

El siguiente ejemplo muestra la sintaxis de la configuración de un puerto UDP adicional:

```
Router(config)# interface fastethernet0/0
Router(config-if)# ip helper-address 192.168.5.100
Router(config-if)# exit
Router(config)# ip forward-protocol udp 4400
Router(config)# no ip forward-protocol udp 69
```

El comando **ip dhcp relay information option** usado con el DHCP Relay habilita el sistema para insertar la opción 82 (*agent DHCP relay information option*) en mensajes **Bootrequest** que son enviados al servidor DHCP. El servidor puede utilizar esta información para asignar la subred correcta.

Configuración de un cliente DHCP

Configurar IOS para la opción del DHCP como cliente es simple.

```
Router(config)# interface fastethernet0/0

Router(config-if)# ip address dhcp
```

Un router puede ser cliente, servidor o ambos a la vez en diferentes interfaces.

RESOLUCIÓN DE FALLOS EN DHCP

En un principio ejecutar un **show run** servirá para ver los parámetros estándares de configuración expuestos anteriormente. Si no se observa dónde está el fallo se puede seguir el siguiente proceso de análisis.

Partiendo de la base de que el cliente DHCP está correctamente configurado y conectado a la red, el servidor DHCP se está ejecutando en el router.

1. Verificar que el cliente no está recibiendo una dirección IP, verificar la conectividad de ambos a la red
2. Verificar que el servidor DHCP esté configurado, como así también que el cliente es capaz de enviar tráfico.
3. Examinar la base de datos DHCP con el comando **show ip dhcp database**.
4. Para ver la configuración mas detallada de DHCP se puede utilizar el comando **show ip dhcp server statistics**.
5. Para mostrar las direcciones IP que han sido asignadas utilizar el comando **show ip dhcp binding**. Para borrar una dirección en la base de datos **clear binding**.
6. Para monitorizar las acciones del servidor DHCP **debug ip dhcp server events** y el **debug ip dhcp server packets**.
7. Finalmente verificar la existencia de listas de acceso que puedan estar denegando el tráfico DHCP.

INTRODUCCIÓN A BGP

BGP (Border Gateway Protocol) es un protocolo de enrutamiento moderno diseñado para ser escalable y poder utilizarse en grandes redes creando rutas estables entre las organizaciones. BGP soporta **VLSM** (Variable Length Subnet Mask), **CIDR** (Classless Interdomain Routing) y sumarización.

BGP es un protocolo de enrutamiento extremadamente complejo, usado entre organizaciones multinacionales y en Internet. El principal propósito de BGP es conectar grandes redes o sistemas autónomos. Las grandes organizaciones utilizan BGP como el vínculo entre diferentes divisiones empresariales. BGP se utiliza en Internet para conectar diferentes organizaciones entre sí.

Es el único protocolo que actualmente soporta enrutamiento entre dominios. Los dispositivos, equipos y redes controlados por una organización son llamados sistemas autónomos, **AS**. Esto significa independencia, es decir, que cada organización es independiente de elegir la forma de conducir el tráfico y no se los puede forzar a cambiar dicho mecanismo. Por lo tanto BGP comunica los AS con independencia de los sistemas que utilice cada organización.

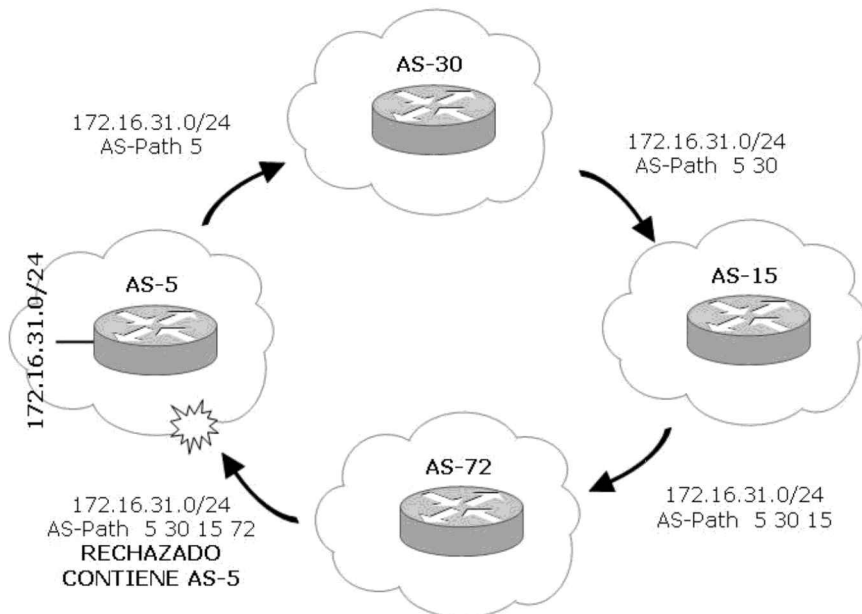
Otro punto clave es que BGP pretende que las redes permanezcan despejadas de tráfico innecesario el mayor tiempo posible. Mientras que los IGP están buscando la última información y ajustando constantemente las rutas acordes con la nueva información que se recibe, BGP está diseñado para que las rutas sean estables y que no se estén advirtiendo e intercambiando constantemente. Las

configuraciones de BGP requieren determinaciones de políticas muy complicadas, de modo que dada la complejidad del protocolo y el inmenso tamaño de la tabla de enrutamiento, que pueden ser cientos de miles, no se puede estar cambiando constantemente decisiones de enrutamiento haciendo que los routers estén constantemente sobrecargados.

Funcionamiento básico de BGP

BGP asocia redes con sistemas autónomos de tal manera que otros router envían tráfico hacia el destino a través de un sistema autónomo. Cuando el tráfico llega a los routers frontera de BGP, es trabajo de los routers del IGP encontrar el mejor camino interno.

BGP es un protocolo **path-vector**, aunque mantiene muchas características comunes con los de vector-distancia. Las rutas son registradas de acuerdo con los sistemas autónomos por donde está pasando y los bucles son evitados rechazando aquellas rutas que tienen el mismo número de sistema autónomo al cual están llegando. La idea de cómo BGP evita los bucles se ilustra en la siguiente figura:



Los vecinos BGP son los llamados *peers*, éstos no son automáticamente descubiertos sino que deben estar predefinidos. Existen cuatro tipos de mensajes en BGP para que la relación sea construida y posteriormente mantenida:

- **Open**
- **Keepalive**
- **Update**
- **Notification**

Cuando el proceso de BGP comienza se crean y mantienen las conexiones entre los peers utilizando el puerto TCP 179 a través de mensajes **BGP open**, posteriormente las sesiones son mantenidas enviando constantemente mensajes **keepalive** y la información del peer se mantiene en una tabla de vecinos separada. Si un peer es reseteado, éste envía un mensaje de notificación para indicar la finalización de la relación. Cuando se establece por primera vez la relación de vecindad, los routers BGP intercambian sus tablas de enrutamiento por completo utilizando mensajes **update**. Finalmente sólo se enviarán actualizaciones incrementales cuando existan cambios en la red.

Jerarquías BGP

Otros protocolos de enrutamiento han sido creados de tal manera que soporten sumalizaciones y para que se pueda organizar la red de manera jerárquica. Las organizaciones no están distribuidas jerárquicamente, por lo tanto BGP debe trabajar con cualquier topología que le sea dada. BGP se beneficia de la sumarización de la misma manera que los demás protocolos de enrutamiento, es decir, menos consumo de recursos de memoria y CPU, y tablas de enrutamiento más pequeñas.

Una red BGP optimizada será altamente sumarizada pero no necesariamente de manera jerárquica. BGP por naturaleza proporciona un resumen de las rutas claves identificando los posibles caminos entre sistemas autónomos. Debido a que los AS no están bien organizados, las redes BGP reflejan esa falta de organización. BGP puede ser implementado entre redes o dentro de una red. BGP detecta los bucles mirando las rutas de los AS-Path.

Cuando utilizar BGP

Debido a su complejidad y especialización para funciones externas, BGP se utiliza para los siguientes casos:

- BGP es el único protocolo de enrutamiento que puede conectar una organización a diferentes sistemas autónomos.

- BGP debería ser considerado si se desea implementar una política de enrutamiento, como por ejemplo controlar el enlace hacia un ISP.
- BGP es el protocolo adecuado para un AS utilizado como AS de tránsito y se interconecte a otros sistemas autónomos. Un ISP es un típico AS de tránsito.

BGP probablemente no sea necesario si algunos de los requerimientos anteriores no se cumplen. Si los requerimientos de enrutamiento que se necesitan pueden llevarse a cabo de una manera más simple, como por ejemplo con una ruta por defecto, no se debería estar pensar en BGP como solución. Los recursos necesarios para un buen funcionamiento de BGP son elevados, por lo tanto otras opciones deben ser siempre analizadas antes de implementarlo, ahorrando de esta forma dinero en dispositivos y configuraciones complejas.

Tablas de BGP

El enrutamiento a través de BGP involucra tres tipos de tablas:

- **Tabla de vecinos**
- **Tabla de BGP**
- **Tabla de enrutamiento IP**

Las rutas de BGP son mantenidas en una tabla de BGP separada y las mejores rutas son pasadas a la tabla de enrutamiento. A diferencia de los protocolos detallados en los capítulos anteriores, BGP no utiliza una métrica. En su lugar BGP emplea un proceso de 10 pasos para seleccionar las rutas dependiendo de una serie de propiedades. Este tema se detallará más adelante y es una de las principales cualidades de BGP. En suma, BGP soporta herramientas como route-maps y listas de distribución que permiten al administrador cambiar el flujo de tráfico basado en los atributos de este protocolo.

CONECTANDO A INTERNET CON BGP

Como protocolo de enrutamiento externo BGP es utilizado para conectar hacia y desde Internet y para enrutar tráfico dentro de Internet. Se debe estar completamente seguro de que la cantidad de tráfico y rutas no saturará la red. Dos consideraciones importantes en el diseño de BGP son la necesidad de enlaces redundantes hacia Internet, llamados *multihoming*, y controlar cuánto tráfico de enrutamiento se quiere recibir desde Internet.

Multihoming

En organizaciones modernas la falta de conectividad a Internet puede llevar a la pérdida de conectividad a determinados servidores y páginas Web hasta la pérdida de telefonía IP. Internet contiene una cantidad enorme y variada de tráfico y de igual manera importante, es por lo tanto una tarea de los administradores poseer una solución a la falta de conectividad hacia Internet. Las conexiones redundantes son la solución a la pérdida de un enlace. Esta solución es conocida como *multihoming*.

La idea del funcionamiento de *multihoming* es que ante el fallo de un enlace active un segundo enlace hacia Internet. Si estos dos enlaces pueden estar activos a la vez proporcionará una mayor efectividad en la conexión a Internet, mayor rendimiento y redundancia. El balanceo de carga en BGP a través de múltiples enlaces o diferentes AS son las políticas de configuración del protocolo.

Multihoming podría efectuar múltiples conexiones al mismo ISP o a ISP diferentes. Conectado a más de un ISP puede generar algunos de los siguientes problemas:

1. El espacio de direccionamiento es advertido de forma incorrecta por el proveedor.
2. Las rutas que se advierten o las rutas externas de las que depende la red son advertidas con diferentes máscaras. Debido a que las más específicas serán siempre utilizadas, no se tendrán en cuenta los atributos de BGP.
3. Cuando la conexión es a más de un ISP, el AS puede llegar a ser un sistema autónomo de tránsito entre esos ISP. Esto es rechazado debido a que no es conveniente que el AS tenga tráfico ajeno al propio sistema autónomo.

Generalmente el ISP es capaz de tratar con este tipo de problemas y configurará su red para protegerse, aun así no se puede confiar plenamente en que su parte esté bien configurada. También es importante el contacto entre los dos ISP implicados para que puedan planificar los cambios necesarios.

Información de enrutamiento desde Internet

Al conectarse a Internet es necesario planificar qué actualizaciones serán enviadas y recibidas desde el mundo exterior.

Existen tres formas de conectarse a Internet:

1. Aceptar sólo rutas por defecto desde todos los ISP. En este caso los consumos de recursos serán muy bajos y la selección de rutas se hará utilizando el router BGP más cercano.
2. Aceptar algunas rutas más las rutas por defecto desde los ISP. En este caso el consumo de recursos de memoria y CPU será medio. El router seleccionará la ruta específica y si no la conoce lo hará a través del router BGP más cercano.
3. Aceptar todas las rutas desde todos los ISP, en este caso el consumo de recursos es alto pero en contra posición siempre se elegirá la ruta más directa.

La manera más fácil de conectarse a Internet es tomar rutas por defecto desde todos los proveedores, proporcionando una ruta redundante en caso de que la principal falle. Si sólo se recibe una ruta de cada proveedor la utilización de memoria y CPU es muy baja. El punto negativo es que no siempre se elegirán los caminos más cortos, el tráfico se dirigirá simplemente hacia el router frontera más cercano. Si las necesidades de la organización no son exigentes, este tipo de conexión es la adecuada.

El caso completamente opuesto al anterior es tomar todas las rutas que llegan desde los proveedores, obteniendo el mejor enrutamiento posible. Sin embargo los recursos consumidos son altos. La tabla de enrutamiento de Internet puede contener 300.000 rutas en constante crecimiento (200.000 rutas consumen unos 200 MB de memoria). La sincronización de la tabla y la recepción de actualizaciones consumen ciclos de procesador y capacidad en la red. Normalmente este tipo de conexión es soportada por los ISP en cuyo caso los dispositivos (Cisco GSR 12000) son capaces de soportar todo el tráfico y las tablas completas con altas velocidades de conexión.

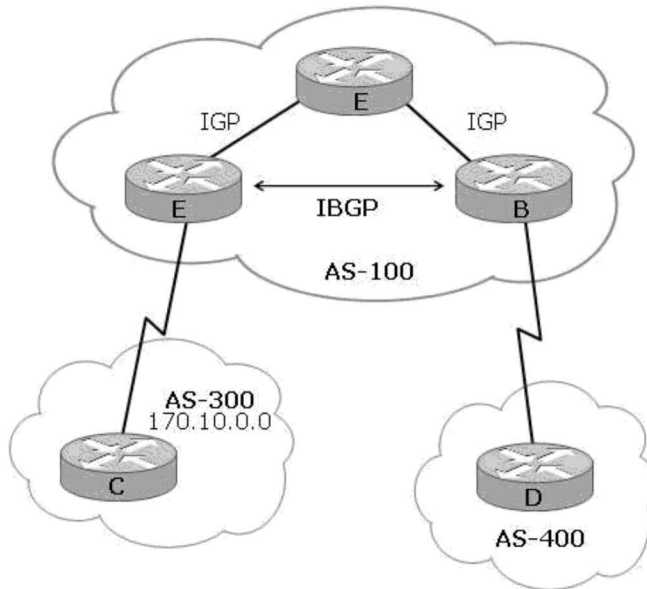
El término medio a estas dos situaciones será emplear algunas rutas específicas y rutas por defecto hacia el proveedor. De esta forma el tráfico que el router conoce tomará el camino más corto y los destinos desconocidos utilizarán las rutas por defecto hacia el router frontera más cercano. La elección de alguna de estas tres opciones es importante pero no permanente, las configuraciones siempre pueden adaptarse a los requerimientos de la organización.

Sincronización

Cuando un AS proporciona un sistema de tránsito para otros AS y tiene router que no son BGP, el tráfico de tránsito podría ser descartado si los routers intermediarios que no ejecutan BGP desconocen esas rutas. La regla de BGP de sincronización dice que si un sistema autónomo proporciona un servicio de tránsito

a otro sistema autónomo, BGP no debería anunciar más rutas hasta que todos los routers dentro de ese AS hayan aprendido acerca de esa ruta vía un IGP.

La siguiente figura muestra un ejemplo de la sincronización:



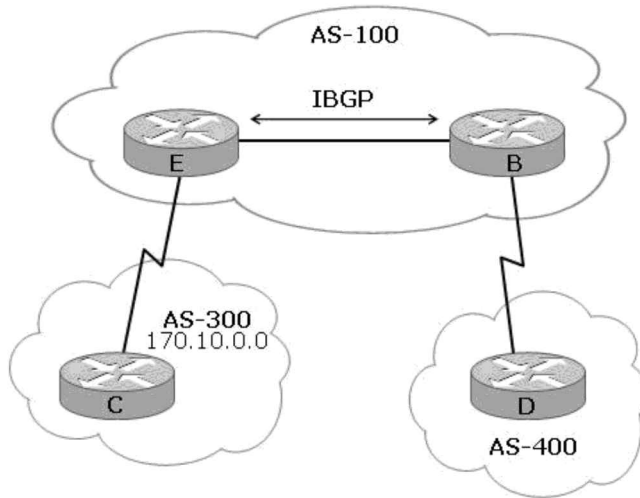
El router C envía actualizaciones acerca de la ruta 170.10.0.0 hacia el router A; el router A y el router B están ejecutando IBGP (Internal BGP). El router B recibe actualizaciones acerca de la red 170.10.0.0 vía IBGP, si el router B quiere alcanzar la red 170.10.0.0, enviará el tráfico hacia el router E. Si el router A no redistribuye la red 170.10.0.0 dentro de un IGP, el router E desconoce que la ruta 170.10.0.0 existe, por lo tanto descartará los paquetes. Si el router B advierte al AS 400 que puede alcanzar dicha red antes de que el router E aprenda sobre dicha red vía un IGP, el tráfico que viene desde el router D hacia el router B con destino 170.10.0.0 pasará por el router E y será descartado.

Esta situación es controlada con la regla de sincronización de BGP, la cual dice que un sistema autónomo, en este caso AS 100, pasa tráfico desde un AS hacia otro y no advertirá una ruta antes de que todos los routers dentro del AS 100 hayan aprendido esa ruta vía un IGP. En este caso el router B espera hasta aprender sobre la red 170.10.0.0 vía un IGP antes de enviar las actualizaciones al router D. Hay casos en los que es preferible deshabilitar la sincronización permitiendo que BGP converja más rápidamente, pero en estos casos pueden producirse pérdidas de paquetes.

Si algunas de las siguientes condiciones se cumplen es posible que sea necesario deshabilitar la sincronización:

- El sistema autónomo no pasa tráfico entre sistemas autónomos.
- Todos los routers de tránsito ejecutan BGP.

La siguiente figura es un ejemplo de una topología donde es posible deshabilitar la sincronización:



La siguiente sintaxis muestra cómo deshabilitar la sincronización dentro del modo de enrutamiento:

```
Router(config-router)# no synchronization
```

En las versiones actuales de IOS la sincronización viene deshabilitada por defecto, para habilitarla se utiliza el siguiente comando:

```
Router(config-router)# synchronization
```

Complementariamente existen otras maneras opcionales de solventar el problema de la sincronización como por ejemplo los routers *reflectors*, las confederaciones o MPLS.

ESTADOS DE BGP

Los estados que toma BGP son los siguientes:

- **Idle**, durante este estado el router esta buscando a los vecinos, técnicamente BGP espera una fase llamada **start**. Este evento puede ser iniciado por un administrador o por el sistema BGP. Un administrador estableciendo una sesión BGP o reseteando una sesión que ya existe causa un evento **start**.
- **Connect**, BGP espera que se complete la conexión del protocolo de transporte, en este caso TCP puerto 179. Si la conexión de TCP se realiza satisfactoriamente, el estado pasa a la fase **open sent**. En el caso de que no sea satisfactoria el estado cambiará a **active**.
- **Active**, intenta establecer una vecindad iniciando la conexión a través del protocolo de transporte. En caso de que lo consiga pasará al siguiente estado, **open sent**. Cuando el temporizador **connect retray** expira BGP lo reinicia y vuelve al estado connect. Si un router permanece entre los estados **connect** y **active** revela que la conexión TCP no se puede establecer. El estado active indica que el router esta intentando iniciar la sesión TCP.
- **Open Sent**, en este estado BGP espera los mensajes **open** del vecino, estos mensajes son chequeados para verificar que los datos son correctos, que las versiones de BGP sean las debidas como así también el número del sistema autónomo.
- **Open Confirm**, BGP espera los mensajes **keepalive**, si recibe estos mensajes de su vecino entonces la sesión pasa al siguiente estado.
- **Established**, es el estado final y el necesario para que BGP comience a funcionar, se intercambien rutas, actualizaciones o keepalives.

Cuando un router permanece todo el tiempo en el estado **idle** será necesario verificar la existencia de un salto hacia el vecino con el que se quiere establecer la vecindad. Si el estado es permanentemente active habrá que verificar la existencia de un firewall que tenga el puerto TCP 179 abierto.

El siguiente ejemplo muestra el estado de un router BGP:

```
Router# show ip bgp neighbors
BGP neighbor is 10.1.1.1, remote AS 100, external
BGP version 4, remote router ID 172.31.2.3
BGP state = Established, up for 00:19:10
```

CONFIGURACIÓN DE BGP

Comandos básicos

BGP ha sido diseñado para conectar diferentes sistemas autónomos entre sí. Los pasos para habilitar BGP consisten en identificar el sistema autónomo e identificar a los vecinos y su correspondiente sistema autónomo. La configuración también debe incluir las redes que se quieren anunciar.

Para iniciar el proceso BGP se utiliza la siguiente sintaxis:

```
Router(config)# router bgp autonomous-system-number
```

A diferencia de muchos otros protocolos BGP sólo puede ejecutar un proceso en cada router. Al intentar configurar más de un proceso BGP el router mostrará el número de proceso que se está ejecutando actualmente.

Identificando vecinos y definiendo peer-groups

BGP no se conecta a otros routers de manera automática, hay que predefinirlos. El comando **neighbor** es utilizado para definir cada uno de los vecinos y su correspondiente sistema autónomo. Si el AS del vecino es el mismo que el local, habrá una conexión **IBGP** (Internal BGP); si el vecino posee un AS diferente, la conexión es **EBGP** (External BGP). Una vez que los vecinos son definidos habrá mas comandos dentro del comando neighbor que se utilizarán para definir las políticas del filtrado de rutas etc.

La siguiente sintaxis muestra la configuración del comando **neighbor**:

```
Router(config-router)# neighbor ip-address remote-as
autonomous-system-number
```

Listar a los vecinos y sus políticas asociadas puede resultar muy pesado. Todos estos grandes bloques de configuración son difíciles de interpretar y de resolver fallos en la red. El concepto del **peer-group** ayuda a solventar estos problemas.

Un peer-group es un grupo de vecinos que comparten la misma política de actualizaciones. Los routers son listados como miembros del mismo peer-group de tal manera que las políticas asociadas con el grupo lo serán también del router. De esta forma las políticas son aplicadas a cada vecino aunque también se pueden definir parámetros individuales personalizando a cada uno de los vecinos, además de aplicar una regla general propia del peer-group.

Los peer-group además de reducir la configuración ayudan a liberar al router de sobrecarga. Si cada vecino es configurado individualmente, entonces el proceso de actualizaciones de BGP tiene que ejecutarse separadamente para cada uno de ellos. Si múltiples vecinos son configurados como un grupo, el proceso de actualizaciones se ejecuta una vez enviando la misma actualización para todos los miembros del grupo. Para que esto resulte adecuadamente los miembros del grupo deben ser internos o externos.

El comando **neighbor peer-group-name** es utilizado para crear el peer-group y asociar a los peer dentro de un sistema autónomo. Una vez que el peer-group ha sido definido los miembros son configurados con el comando **neighbor peer-group**.

```
Router(config-router)# neighbor peer-group-name peer-group
Router(config-router)# neighbor ip-address | peer-group-name
remote-as autonomoussystem-number
Router(config-router)# neighbor ip-address peer-group peer-
group-name
```

La siguiente sintaxis muestra la configuración básica con tres vecinos:

```
Router(config)# router bgp 200
Router(config-router)# neighbor 190.55.5.2 remote-as 200
Router(config-router)# neighbor 190.55.5.3 remote-as 200
Router(config-router)# neighbor 190.55.5.4 remote-as 200
```

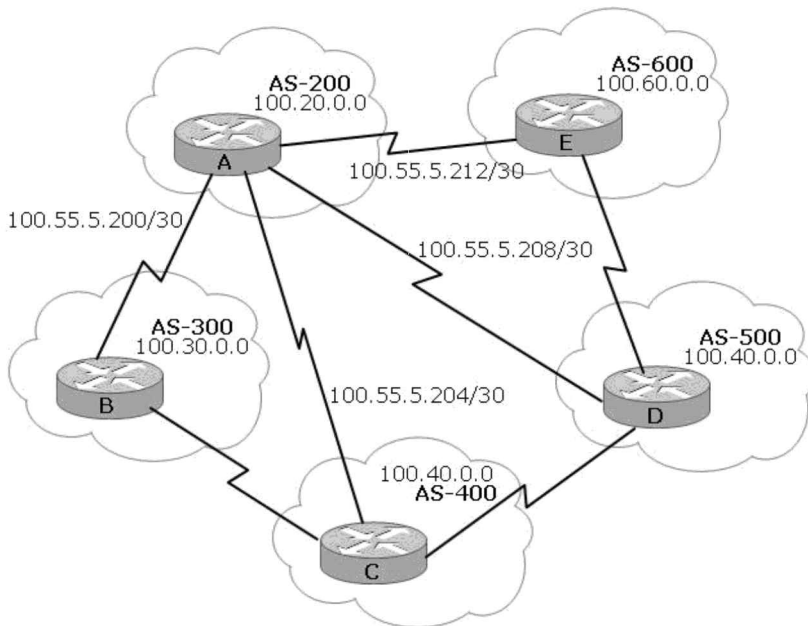
El siguiente es un ejemplo de configuración de peer-group:

```
Router(config)# router bgp 200
Router(config-router)# neighbor Nom-AS peer-group
Router(config-router)# neighbor 190.55.5.2 peer-group Nom-AS
Router(config-router)# neighbor 190.55.5.3 peer-group Nom-AS
Router(config-router)# neighbor 190.55.5.4 peer-group Nom-AS
Router(config-router)# neighbor Nom-AS remote-as 200
```

El comando **neighbor** también puede ser utilizado para deshabilitar un peer. Normalmente un peer es deshabilitado durante una ventana de mantenimiento o para prevenir caídas y subidas en un enlace.

```
Router(config-router)# neighbor ip-address | peer-group-name
shutdown
```

El ejemplo que sigue muestra la configuración básica de los comandos requeridos para que BGP funcione entre sistemas autónomos. Según muestra la topología el router A en el AS-200 está conectado a los routers en los AS-300, AS-400, AS-500 y AS-600.



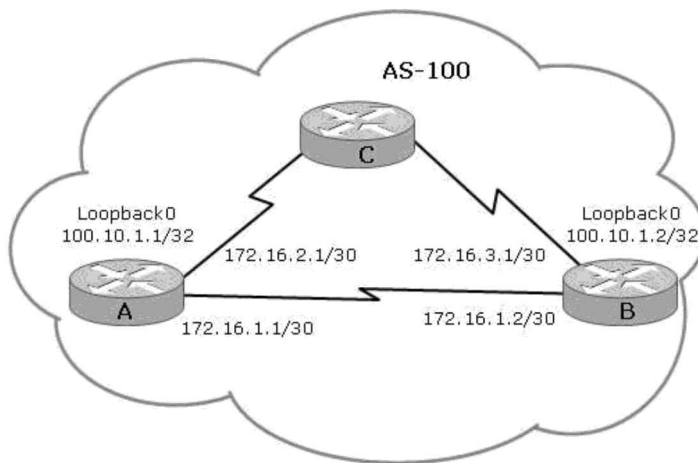
```

RouterA(config)# interface Serial0/0.1
RouterA(config-int)# ip address 190.55.5.201 255.255.255.252
!
RouterA(config)# interface Serial0/0.2
RouterA(config-int)# ip address 190.55.5.205 255.255.255.252
!
RouterA(config)# interface Serial0/0.3
RouterA(config-int)# ip address 190.55.5.209 255.255.255.252
!
RouterA(config)# interface Serial0/0.4
RouterA(config-int)# ip address 190.55.5.213 255.255.255.252
!
RouterA(config)# router bgp 200
RouterA(config-router)# neighbor 190.55.5.202 remote-as 300
RouterA(config-router)# neighbor 190.55.5.206 remote-as 400
RouterA(config-router)# neighbor 190.55.5.210 remote-as 500
RouterA(config-router)# neighbor 190.55.5.214 remote-as 600
```

Dirección IP de origen

Los vecinos de BGP interno no tienen que estar directamente conectados, normalmente un IGP es responsable del enrutamiento dentro del AS de tal forma que los routers que ejecutan BGP pueden alcanzarse mutuamente a través del enrutamiento proporcionado por el IGP.

En la siguiente figura el router A tiene varias direcciones IP. Los routers A y B forman paridad y se encuentran directamente conectados por medio de la red 172.16.1.0, pero si el enlace se cae la interfaz del router B dejaría de responder, incluso cuando el router B está directamente conectado al router C, no sabría que hacer debido a que la dirección configurada para el peer está caída.



Sin embargo si el *peering* se establece mediante direcciones de loopback el problema puede solventarse porque a pesar de que el enlace podría caerse, el router B enviará el tráfico a través del router C. Para la configuración de este ejemplo hay que tener en cuenta que se han utilizado las direcciones de loopback en el IGP.

Otro problema que surge es que cuando BGP crea tráfico la dirección IP de origen es la de la interfaz de salida, en el router B el tráfico hacia el router A tomaría el camino directo desde la interfaz 172.16.1.2, pero si el enlace falla el tráfico redireccionará desde la dirección IP 175.16.3.1. Esta nueva dirección de origen no encaja con la definida como peer, por lo tanto la conexión será rechazada. Para permitir caminos redundantes el vínculo de origen tiene que ser el mismo que el vecino espera. La sintaxis del comando en la siguiente:

```
Router(config-router)# neighbor ip-address | peer-group-name
update-source interface
```

La configuración de los routers A y B está en la siguiente sintaxis:

Router A :

```
RouterA(config)# router bgp 100
RouterA(config-router)# neighbor 100.10.1.2 remote-as 100
RouterA(config-router)# neighbor 100.10.1.2 update-source
loopback0
```

Router B :

```
RouterB(config)# router bgp 100
RouterB(config-router)# neighbor 100.10.1.1 remote-as 100
RouterB(config-router)# neighbor 100.10.1.1 update-source
loopback0
```

Ambos routers establecen el peering hacia las direcciones de loopback de sus vecinos y originan tráfico desde sus propias loopback permitiendo de esa manera redundancias.

Los routers BGP externos normalmente no utilizan ningún protocolo de enrutamiento haciendo peering con direcciones directamente conectadas. Por defecto los paquetes EBGp poseen un valor de TTL igual a uno, lo que previene que puedan viajar a más de un salto. Sería posible crear un modelo de redundancia en el ejemplo anterior, pero es necesario que se cumplan las siguientes condiciones:

- Tiene que existir una ruta hacia ambas direcciones por donde se establecerá el peer.
- Debe ajustarse el TTL para que sea capaz de atravesar el número necesario de saltos.

Para modificar el TTL por defecto en las conexiones externas se utiliza el siguiente comando:

```
Router(config-router)# neighbor ip-address | peer-group-name
ebgp-multihop hops
```

Forzando la dirección del próximo salto

Un *next hop* de BGP es el punto de entrada al sistema autónomo. Normalmente el próximo salto es la dirección del router frontera, pero hay diversas situaciones donde debe cambiarse. Las rutas recibidas desde vecinos externos serán advertidas con próximo salto, con una dirección externa los vecinos internos

podrían no reconocerla como una dirección válida. El comando **next-hop-self** permite al router sustituir su dirección interna de tal manera que los vecinos internos entenderán cómo alcanzar la dirección del próximo salto. La siguiente sintaxis muestra la configuración de este comando:

```
Router(config-router)# neighbor {ip-address | peer-group}  
next-hop-self
```

Definiendo las redes que serán anunciadas

El comando **network** configura las redes que van a ser originadas por este router. Hay una diferencia notable entre el uso del comando **network** en BGP y en otros protocolos de enrutamiento. Con este comando no se identifican las interfaces que van a ejecutar BGP, sino que se identifican las redes que se van a propagar. Se pueden utilizar múltiples comandos **network**, tantos como se requiera. El parámetro de la máscara es muy útil en BGP puesto que puede funcionar con subnetting y supernetting. La sintaxis del comando es la siguiente:

```
Router(config-router)# network network-number mask network-mask
```

Agregación de rutas

Para agregar o sumarizar rutas en un dominio de BGP se utiliza el siguiente comando:

```
Router(config-router)# aggregate-address ip-address mask  
[summary-only] [as-set]
```

En el caso de que se configure el parámetro **summary-only** todas las rutas más específicas serán eliminadas propagándose solamente la sumarización de éstas. Esto es opcional dependiendo de la topología existente pero aun así es conveniente enviar la sumarización para evitar enviar demasiados prefijos. Cuando se utiliza el parámetro **as-set** todos los sistemas autónomos que han sido atravesados serán almacenados en el mensaje update.

El siguiente ejemplo muestra la configuración de BGP utilizando el comando **aggregate-address**, con el parámetro **summarization**. Las rutas con un prefijo mayor que /16 serán sumarizadas. De esta forma la ruta 10.10.35.8/29 no es vista en las actualizaciones *update* debido a que está sumarizada dentro de la red 10.20.0.0/16. Cualquier otra ruta que no esté sumarizada dentro de esa actualización será listada individualmente. La sumarización reduce sobrecarga en la red y simplifica la administración de la misma.

```
Router(config)# interface Serial0
Router(config-int)# ip address 10.255.255.201 255.255.255.224
Router(config)# router bgp 100
Router(config-router)# network 10.10.35.8 255.255.255.252
Router(config-router)# network 10.10.27.0 255.255.255.0
Router(config-router)# network 10.10.100.0 255.255.255.0
Router(config-router)# aggregate-address 10.10.0.0
255.255.0.0 summary-only
Router(config-router)# neighbor 10.255.255.202 remote-as 200
Router(config-router)# neighbor 10.255.255.202 next-hop-self
Router(config-router)# neighbor 10.255.255.206 remote-as 300
Router(config-router)# neighbor 10.255.255.206 next-hop-self
```

Autenticación

La autenticación es una parte importante en BGP sobretodo para proveedores de servicios (ISP). Sin autenticación estos ISP estarían expuestos a múltiples ataques desde Internet. La autenticación de BGP consiste en abordar una clave o contraseña entre los vecinos de tal manera que se envíe un *hash MD5* con cada paquete BGP. Esta autenticación es muy simple de configurar y asocia la contraseña con el vecino:

```
Router(config-router)# neighbor ip-address password password
```

VERIFICACIÓN DE BGP

Los comandos show para BGP brindan una información clara acerca de las sesiones de BGP y de las opciones de enrutamiento:

- **show ip bgp**, muestra la tabla de enrutamiento de BGP.
- **show ip bgp summary**, muestra el estado de las sesiones de BGP y también el número de prefijos aprendidos en cada sesión.
- **show ip bgp neighbors**, muestra la información de la conexión TCP hacia los vecinos, así como el tipo y número de mensajes BGP que están intercambiándose con cada vecino. Cuando la conexión se establece los vecinos intercambian actualizaciones.
- **show processes cpu**, muestra los procesos activos y se usa para identificar los procesos que consumen demasiados recursos, pueden ordenarse por cantidad de recursos consumidos.

Otro comando de gran utilidad es el debug, como todos estos comandos debe utilizarse con la debida precaución. Mostrará la información real de los eventos que se están ejecutando:

```
Router# debug ip bgp [dampening | events | keepalives | updates]
```

El *Route dampening* es un mecanismo para identificar la inestabilidad causada por caídas aleatorias de rutas o flapping. Cuenta la cantidad de veces que una ruta determinada cae aplicando una penalización a cada una de estas caídas. De esta manera BGP puede ignorar esas rutas para utilizar las rutas más estables.

Reestableciendo la vecindad

Luego de configurar cambios en BGP a veces es necesario hacer un *hard reset* que significa resetear la sesión TCP entre ambos vecinos. Se ejecuta con el siguiente comando:

```
Router(config-router)# clear ip bgp { * | address } [soft [in|out]]
```

Este comando desconecta la sesión entre vecinos y la reestablece con la nueva configuración que ha sido aplicada. BGP solamente intercambia rutas cuando la relación de vecindad ha sido establecida. A partir de ese momento si los vecinos cambian las políticas, como por ejemplo una nueva lista de distribución, el único medio para propagar los cambios sin reiniciar el router es resetear la sesión TCP.

Este reinicio puede generar algunos de estos problemas:

- El reinicio de la sesión tarda bastante tiempo en reestablecerse e interrumpe el tráfico mientras se está iniciando nuevamente.
- Al resetear la sesión los routers lo pueden detectar como una caída de un enlace, pudiendo causar que los pares se desasocien.
- Al resetear la sesión la tabla de enrutamiento debe ser enviada por completo nuevamente, generando mucho tráfico.

Existe una alternativa al *hard reset* que es un reinicio de sesión menos violento. El *soft reset* no interrumpe el enrutamiento ni causa flapping. Pueden configurarse tanto para conexiones entrantes como salientes.

La siguiente sintaxis muestra la configuración de un *soft reset* de salida:

```
Router(config-router)# clear ip bgp { * | address } soft out
```

Un *soft reset* le indicará al router local que debe enviar su tabla de enrutamiento BGP completa hacia el vecino pero haciéndolo de tal manera que la sesión TCP que hay entre ambos no caiga. De esta forma los cambios en la política de enrutamiento en dirección hacia el vecino saliente son enviados.

Hay casos en los que se requiere que el vecino envíe sus anuncios siguiendo este sistema, pero la única manera de hacerlo es a través del administrador para que ejecute dicho comando en su router.

El reseteo suave de entrada permite simular la recepción de una actualización. Para poder ejecutarlo en el router local debe existir una copia no procesada de la actualización del vecino.

El comando **soft-reconfiguration-inbound** debe estar pre-configurado para el vecino, como se expone en la siguiente sintaxis. El router guardará dos copias de ese anuncio, antes y después de procesar:

```
Router(config-router)# neighbor ip-address soft-reconfiguration inbound
```

Asumiendo que esta capacidad está disponible, se podrá repetir un anuncio previamente guardado, por ejemplo se podría forzar una actualización desde el vecino a través de los nuevos filtros utilizando el siguiente comando:

```
Router(config-router)# clear ip bgp ip-address soft in
```

Cada vez que se utilice el comando **soft** se le está indicando al router que no cierre la sesión TCP.

ATRIBUTOS DE BGP

Las rutas aprendidas vía BGP llevan asociados ciertos atributos que sirven para determinar qué rutas son la mejor opción hacia un destino y si existen varios caminos hacia ese destino en particular. Para diseñar redes robustas con BGP hay que comprender cómo los atributos de BGP influyen en el proceso de selección de rutas.

La siguiente tabla muestra los atributos BGP soportados en IOS:

Atributo	Categoría	Descripción
Aggregator	Optional, transitive	ID y AS del router que lleva a cabo la sumarización. No se usa en el proceso de selección de caminos.
AS_Path	Well-known, mandatory	Lista de todos los AS a través de los cuales ha pasado. Se prefiere el más corto.

Atomic aggregate	Well-known, discretionary	Al generar una sumarización envía los AS de las rutas que componen dicha sumarización. No se usa en el proceso de selección de caminos.
Cluster ID	Optional, nontransitive	Identifica un clúster, Route Reflectors. No se usa en el proceso de selección de caminos.
Community	Optional, transitive	Etiqueta prefijos. No se usa en el proceso de selección de caminos.
Local preference	Well-known, discretionary	Indica el nivel de preferencia para alcanzar prefijos externos a través de los routers internos. Se prefiere el valor más alto.
Multiple Exit Discriminator (MED)	Optional, nontransitive	Indica a vecinos externos qué camino usar para alcanzar prefijos.
Next Hop	Well-known, mandatory	Peer externo en el AS vecino. No se usa en el proceso de selección de caminos.
Origin	Well-known, mandatory	Código de origen. Preferidos en este orden: (i) IGP, (e) EGP, (?) Incomplete.
Originator ID	Optional, nontransitive	Identifica al Route Reflector. No se usa en el proceso de selección de caminos.
Weight	Optional, not communicated to peers	Propietario de Cisco. Se prefiere el más alto.

Es posible que BGP pueda recibir varios anuncios para la misma ruta desde diferentes orígenes. BGP solamente selecciona un camino como el más adecuado; una vez que dicho camino es seleccionado BGP lo pone en la tabla de enrutamiento y lo propaga a sus vecinos.

El proceso de selección de rutas completo de BGP sigue el siguiente orden:

1. Se prefiere el camino con el **weight** más alto. Este atributo es propietario de Cisco y es local al router en el que se configura.

2. Se prefiere el camino con **local_pref** más alto. Por defecto toma un valor de 100 pero es posible modificarlo.
3. Se prefieren caminos originados localmente vía el comando **network** o **aggregate**, o mediante redistribución desde un IGP. Los caminos locales originados por el comando **network** o **redistribute** son preferidos sobre los localmente agregados usando el comando **aggregate-address**.
4. Se prefiere el camino con el **as_path** más corto.
 - Este paso es omitido si el comando **bgp bestpath as-path ignore** está configurado.
 - Un **as_set** cuenta como 1, no importa cuántos AS haya en el “set”.
 - Los **as_confed_sequence** y **as_confed_set** no están incluidos en la longitud del **as_path**.
5. Se prefiere el camino con el **origin** más bajo, donde IGP<EGP.
6. Se prefiere el camino con la **med** más baja. Por defecto es 0.
7. Se prefieren caminos EBGp sobre caminos IBGP. Si se trata de un camino EBGp continúa en el paso 9.
8. Se prefiere el camino a través del IGP con la menor métrica hacia el *next hop* de BGP
9. Determinar si se requiere instalar múltiples caminos en la tabla de enrutamiento para BGP Multipath.
10. Cuando ambos caminos son externos se prefiere el que fue recibido primero (el más antiguo). Esto minimiza el flapping de rutas. Hay que omitir este paso si se cumple alguna de las siguientes condiciones:
 - Está habilitado el comando **bgp best path compare-router-id**.
 - El router ID es el mismo para múltiples caminos porque las rutas fueron recibidas desde el mismo router.
 - No existe un “best path” actualmente.
11. Se prefiere la ruta que viene desde el router BGP con el router ID más bajo. En caso de que el camino contenga atributos de Route Reflectors RR el originator ID es substituido por el router ID en el proceso de selección de rutas de BGP.
12. Si el *originator* o router ID es el mismo para múltiples caminos, preferimos el camino con la lista de clúster de menor longitud. Esto solamente está presente en entornos con RR.

13. Se prefiere el camino que viene desde el vecino con la dirección IP más baja correspondiente a la usada durante el establecimiento de la sesión TCP, es decir, la que figura en la configuración del vecino.

Existen tres conceptos fundamentales para la configuración de BGP

- La regla de sincronización
- Comportamiento de horizonte dividido en BGP
- La selección de caminos BGP

Controlando la selección de caminos de BGP

El propio sistema que BGP utiliza para controlar la selección de rutas es bastante sofisticado habrá momentos en que el administrador tendrá que influir en ese comportamiento modificando los atributos del protocolo. Existen muchos métodos para influenciar la selección de caminos en BGP, pero los route-maps son los más utilizados.

Los cuatro principales atributos que se suelen modificar son Weight, Local-Preference, MED y *AS-path prepending*. Los dos primeros se modifican para influenciar cómo sale el tráfico del AS y los siguientes indicarán a otros AS cómo se prefiere que ellos alcancen al AS local.

Uso del atributo Weight

Este atributo es propietario de Cisco, selecciona la interfaz de salida cuando existen múltiples rutas hacia un mismo destino. Cuanto más alto es el valor, mayor preferencia. Este atributo es local al router y no se propaga hacia los demás routers. Este atributo es extremadamente potente, porque es el primero que BGP utiliza en el proceso de selección y toma preferencia sobre cualquier otro.

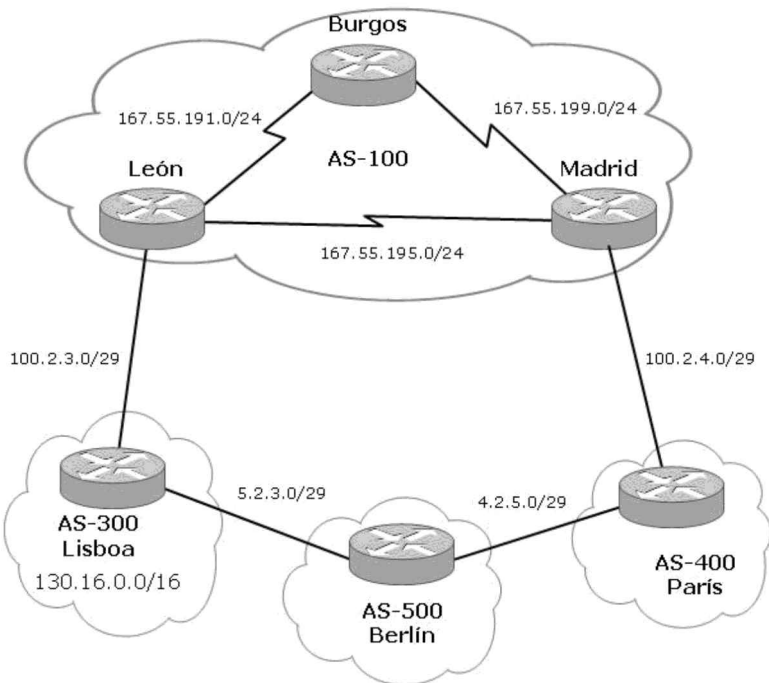
Para configurarlo se utiliza el siguiente comando:

```
Router(config-router)# neighbor {ip-address | peer-group-name} weight weight
```

El Weight puede tomar valores entre 0 y 65535, por defecto es 0 a menos que el router esté originando la ruta, en cuyo caso el peso por defecto será 32768.

La figura que sigue ilustra el uso del atributo Weight y la sintaxis muestra cómo es elegida la ruta a través del router **Madrid**. El Weight ha sido modificado en el router **Burgos** haciendo preferible el camino a través del router **Madrid** sin importar cuál sea la red que se está tratando de alcanzar.

El mejor camino para llegar a la red 130.16.0.0 es a través del router **León**, pero finalmente y por los cambios efectuados será a través de **Madrid**.



```
Burgos(config)# router bgp 100
Burgos(config-router)# bgp log-neighbor-changes
Burgos(config-router)# neighbor 167.55.191.3 remote-as 100
Burgos(config-router)# neighbor 167.55.199.2 remote-as 100
Burgos(config-router)# neighbor 167.55.199.2 weight 200
```

Uso del atributo Local-Preference

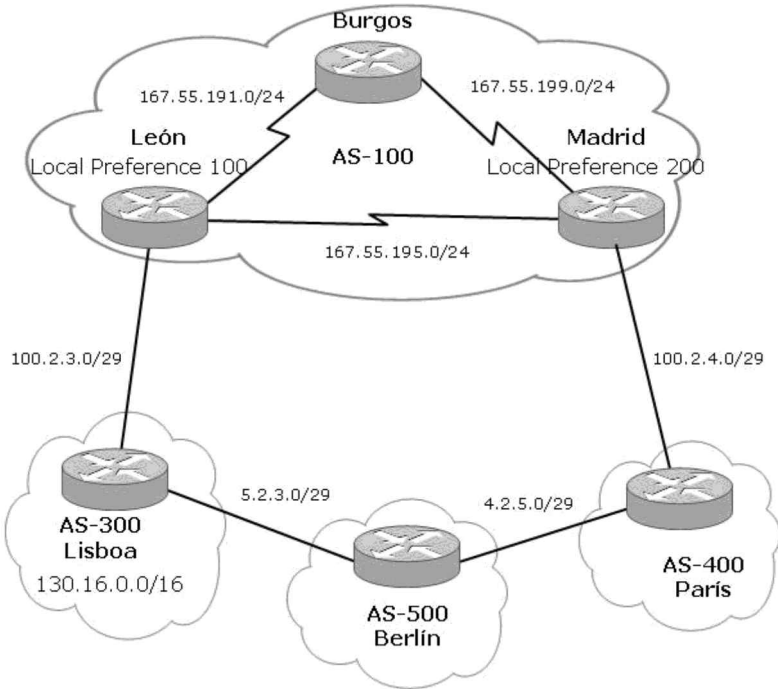
Este atributo es fácil de configurar: puede hacerlo por defecto o por prefijo. La sintaxis es la siguiente:

```
Router(config-router)# bgp default local-preference value
```

El atributo Local-Preference tiene un rango entre 0 y 4294967295, cuanto más alto el valor mayor preferencia. Por defecto lleva un valor de 100.

El siguiente ejemplo está basado en la siguiente figura, el atributo Local-Preference está configurado en el router **Madrid** con un valor de 200 y se propaga en todas las actualizaciones a todos los peers.

El valor de Local-Preference esta configurado a 100 en el router **León** y se propaga a todos los peers. Cuando el router **Burgos** deba decidir qué ruta utilizar para llegar a la red 130.16.0.0 el Local-Preference más alto dicta que **Madrid** es la mejor salida desde el AS.



```
Madrid(config)# router bgp 100
!
Madrid(config-router)# bgp default local-preference 200
Madrid(config-router)# aggregate-address 167.55.0.0
255.255.0.0 summary-only
Madrid(config-router)# neighbor 100.2.4.4 remote-as 400
Madrid(config-router)# neighbor 100.2.4.4 default-originate
!
Madrid(config-router)# neighbor 167.55.195.3 remote-as 100
Madrid(config-router)# neighbor 167.55.199.1 remote-as 100
Leon(config)# router bgp 100
!
Leon(config-router)# bgp default local-preference 100
Leon(config-router)# network 167.55.0.0
Leon(config-router)# aggregate-address 167.55.0.0 255.255.0.0
summary-only
Leon(config-router)# neighbor 100.2.3.2 remote-as 300
Leon(config-router)# neighbor 167.55.191.1 remote-as 100
Leon(config-router)# neighbor 167.55.195.2 remote-as 100
```

La Local-Preference también puede ser aplicada por prefijos. Aunque existen varios métodos para modificar las rutas los route-maps son los más utilizados. Para cambiar los parámetros de los route-maps simplemente se modifica el parámetro en el comando neighbor:

```
Router(config-router)# neighbor {IP | peer group} route-map  
route-map name {in|out}
```

La siguiente sintaxis muestra un route-map aplicada a todas las rutas redistribuidas desde un peer. Las rutas son comparadas con una ACL y las que son permitidas adquieren una Local-Preference de 500. La línea 20 no tiene sentencia **match** por lo tanto aplica a todo el resto de las rutas. Las rutas denegadas adquieren un Local-Preference de 50.

```
Lisboa(config)# router bgp 100  
Lisboa(config-router)# neighbor 10.5.5.25 remote-as 100  
Lisboa(config-router)# neighbor 10.5.5.25 route-map example_LP in  
Lisboa(config-router)# exit  
Lisboa(config)# access-list 5 permit 10.1.1.0 0.0.0.255  
Lisboa(config)# route-map example_LP permit 10  
Lisboa(config-rmap)# match ip address 5  
Lisboa(config-rmap)# set ip local-preference 500  
Lisboa(config)# route-map example_LP permit 20  
Lisboa(config-rmap)# set ip local-preference 50
```

Uso del atributo MED

Este atributo es advertido a los vecinos externos para influenciarlos en el sentido de cómo van a poder alcanzar a nuestro sistema autónomo, es decir, que se modifica el tráfico de entrada hacia nuestro AS. Por ejemplo cuando existen enlaces redundantes hacia un ISP y se necesita que el tráfico fluya sobre el enlace con mayor capacidad. Con el Weight o con el Local-preference se puede controlar cómo los routers locales envían tráfico fuera del sistema autónomo, pero los vecinos externos no reciben esa información. Si se configura el atributo MED para que sea más bajo en uno de los caminos, éste será elegido por los vecinos para enviar tráfico hacia el AS local.

MED es un atributo opcional y transitivo, cuanto menor sea tendrá mayor preferencia, por defecto el valor de MED es 0.

La sintaxis de configuración del atributo MED es la siguiente:

```
Router(config-router)# default-metric value
```

El ejemplo que sigue muestra un route-map anunciado hacia un peer. En primer lugar las rutas son comparadas con una ACL y las rutas permitidas llevarán una métrica de 500, la línea 20 del route-map no incluye una sentencia match por lo tanto todas las demás rutas que sean denegadas por la ACL llevarán una métrica de 5000.

```
Router(config)# router bgp 100
Router(config-router)# neighbor 10.6.6.36 remote-as 200
Router(config-router)# neighbor 10.6.6.36 route-map med_rm out
Router(config-router)# exit
Router(config)# access-list 5 permit 10.1.1.0 0.0.0.255
Router(config)# route-map med_rm permit 10
Router(config-rmap)# match ip address 5
Router(config-rmap)# set metric 500
Router(config)# route-map med_rm permit 20
Router(config-rmap)# set metric 5000
```

Uso del atributo AS-path

El atributo **AS-path** prepending es del tipo Well-known y obligatorio para todas las actualizaciones. Es el mecanismo que BGP utiliza para detectar bucles de enrutamiento y que además es el sistema para decidir que ruta es la más corta. Mientras más AS sean atravesados peor será el camino, siempre se utilizará el AS-path más corto.

EL AS-path que se envía a un vecino puede modificarse utilizando un route-map de esta forma se puede replicar el AS local para conseguir establecer caminos menos preferidos que otros.

```
Router(config)# router bgp 65005
Router(config-router)# neighbor 193.0.0.2 remote-as 65100
Router(config-router)# neighbor 193.0.0.2 route-map prepend out
Router(config-router)# route-map PREPEND permit 10
Router(config-router)# set as-path prepend 65005 65005 65005
```

VERIFICACIÓN DE LOS ATRIBUTOS

El comando **show ip bgp** muestra los atributos, los valores y el estado. Es un buen comando para verificar la configuración y los cambios efectuados y para administrar el flujo del tráfico desde y hacia nuestro AS.

Los siguientes ejemplos muestran dos **show ip bgp** basados en los casos anteriores, observe el cambio del Weight a 200 en el segundo ejemplo y su efecto en la red.

Chicago# **show ip bgp**

BGP table version is 22, local router ID is 192.168.0.231

Status codes: s suppressed, d damped, h history, * valid, > best,
i - internal, r RIB-failure

Origin codes: i - IGP, e - EGP, ? - incomplete

Network	Next Hop	Metric	LocPrf	Weight	Path
*>i4.0.0.0	100.2.4.4	0	100	0 400	I
*>i5.0.0.0	100.2.3.2	0	100	0 300	I
*>i100.2.3.0/29	100.2.3.2	0	100	0 300	I
*>i100.2.4.0/29	100.2.4.4	0	100	0 400	I
*>i130.16.0.0	100.2.3.2	0	100	0 300	I
r>i167.55.0.0	167.55.191.3	281600	100	0	I

Chicago# **show ip bgp**

BGP table version is 8, local router ID is 192.168.0.231

Status codes: s suppressed, d damped, h history, * valid, > best,
i - internal, r RIB-failure

Origin codes: i - IGP, e - EGP, ? - incomplete

Network	Next Hop	Metric	LocPrf	Weight	Path
*>i4.0.0.0	100.2.4.4	0	100	200 400	i
*>i5.0.0.0	100.2.3.2	0	100	0 300	i
*>i100.2.3.0/29	100.2.3.2	0	100	0 300	i
*>i100.2.4.0/29	100.2.4.4	0	100	200 400	i
*>i130.16.0.0	100.2.4.4		100	200 400 500	300 i
* i	100.2.3.2	0	100	0 300	i
r>i167.55.0.0	167.55.191.3	281600	100	0	i

Campo	Descripción
BGP table version	Número interno de la versión de la tabla. Se incrementa cuando la tabla cambia.
local router ID	La dirección IP usada como BGP ID.
status codes	El estado se muestra al principio de cada línea y puede tomar estos valores: • s - suprimido • * - válido • > - mejor entrada para esa red • i - aprendida via iBGP • d - dampening ha sido aplicado

Origin	El código de origen está situado al final del <i>AS-path</i>. Puede ser uno de los siguientes valores: • i - La red ha sido advertida usando el comando network. • e - La red ha sido originada desde EGP (actualmente no se usa). • ? - La red ha sido redistribuida en BGP.
Network	Un prefijo.
Next hop	IP del primer vecino externo en el camino. Si es 0.0.0.0 indica que la ruta ha sido auto originada.
Metric	Atributo de BGP usado para influenciar el tráfico de vuelta. Por defecto es 0.
LocPrf	Atributo de BGP usado para indicar al AS qué router usar para salir hacia otro AS. Por defecto es 100.
Weight	Atributo de BGP para indicar al router qué camino tomar para llegar al destino. Es propietario de Cisco y es local al router.
Path	Sistemas Autónomos que hay hacia el destino. Puede haber una entrada en este campo por cada AS que haya que atravesar.

MULTICAST

INTRODUCCIÓN A MULTICAST

Tipos de direcciones IP

En una red IP pueden existir tres tipos de direccionamiento:

- **Unicast**, los paquetes son enviados desde un origen hacia un sólo destino.
- **Broadcast**, los paquetes son enviados con un direccionamiento de difusión.
- **Multicast**, los paquetes son enviados a una dirección especial de grupo.

Unicast: los paquetes unicasts son enviados a una dirección específica. Cuando se envía información a un determinado grupo será necesario hacerlo a cada uno de los componentes de dicho grupo, lo que provoca un consumo de ancho de banda. Unicast es enviado por un router o por un switch de capa 3 que encuentra en su tabla de direccionamiento la dirección IP. Un switch de capa 2 confía en la dirección MAC del destino. El envío unicast está habilitado por defecto, es el más común de los tres tipos de direccionamiento.

Broadcast: es la manera de comunicar exactamente la misma información a todo un grupo. Los broadcasts son transmitidos al medio y recibidos por todos los dispositivos. Existen protocolos de enrutamiento que utilizan broadcasts para distribuir la información de enrutamiento. En lugar de requerir varios paquetes

unicast simplemente se envía un paquete que alcanza a todos los dispositivos. Los broadcasts son útiles además, para cuando se quiere enviar una información a un destino desconocido. DHCP utiliza broadcast para asignar direcciones unicast.

Las NIC están programadas para escuchar todo el tráfico y de esa manera reconocer el tráfico que está destinado a la propia dirección local MAC o la dirección MAC de broadcast y enviar las tramas a las capas superiores.

El problema de los broadcasts es que deben ser procesados por la CPU, por ejemplo, un PC que recibe una actualización de RIP tendrá que procesar el paquete para comprender que no está interesado en el mismo.

Los broadcasts en ethernet son enviados a la dirección MAC reservada ffff.ffff.ffff. Los switches de capa 2 inundan los broadcasts fuera de todos los puertos en la misma VLAN. Los broadcasts en IP están reservados en la dirección 255.255.255.255. Los routers no retransmiten tráfico broadcast por defecto.

Multicast: son direcciones de grupo. Un dispositivo IP se une a un grupo reconociendo una dirección IP de otro grupo y reprogramando su tarjeta de red (NIC) para copiar todo el tráfico destinado a la dirección MAC del grupo. Debido a que el tráfico multicast está dirigido a diferentes MAC algunos host prestarán atención mientras que otros lo ignorarán. EIGRP utiliza 224.0.0.10 y la dirección MAC 0100.5E00.000A, los routers prestarán atención a este tráfico pero un PC lo ignorará.

Debido a que multicast permite discriminar el tráfico a nivel de la tarjeta de red, es decir, a nivel de enlace de datos, se pierde menos tiempo de procesamiento de dicho tráfico.

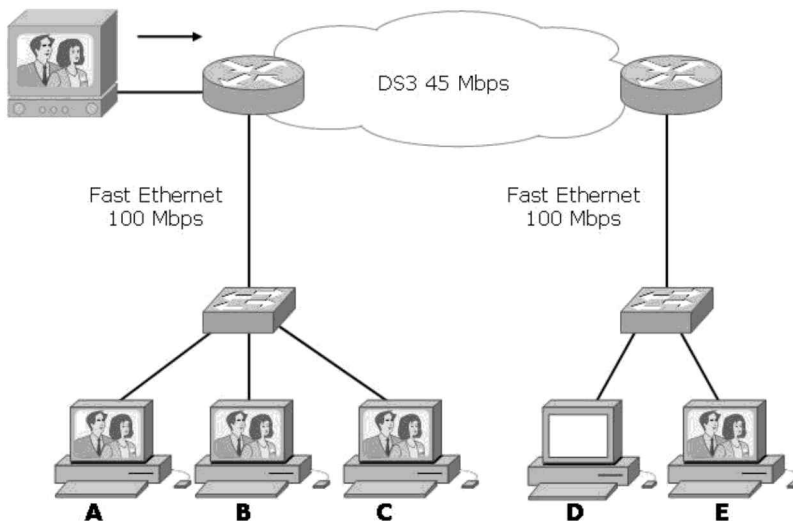
El tráfico multicast generalmente es unidireccional, hay un origen que envía el tráfico a todos los destinos, mientras que éstos devuelven el tráfico de manera unicast. El tráfico multicast es enviado utilizando el protocolo UDP, por lo tanto no está orientado a la conexión de tal manera que no hay verificación de que los paquetes sean realmente entregados, dejando esta responsabilidad a los destinos o la aplicación.

Los host que quieren recibir datos de una dirección origen multicast pueden unirse o dejar el grupo de forma dinámica habilitando la dirección MAC del grupo en su tarjeta de red. Un host puede ser miembro de más de un grupo multicast al mismo tiempo.

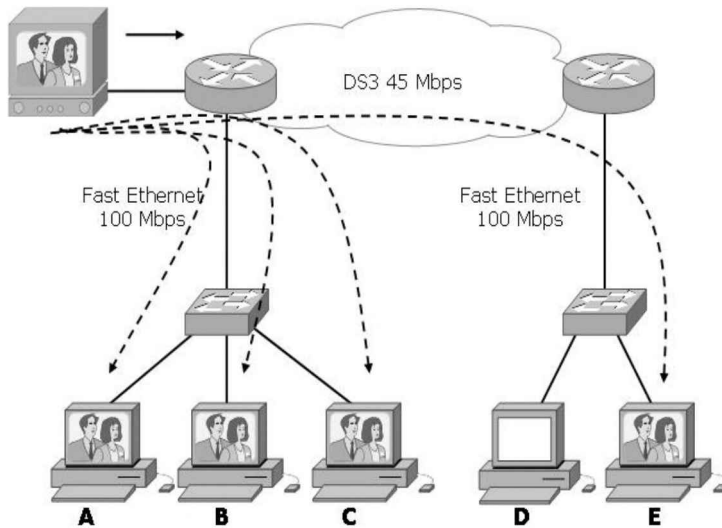
El tráfico multicast solamente es procesado por los host que están programados para recibirlo, un router por defecto no reenvía paquetes multicast a no ser que esté habilitado para hacerlo. Por defecto los switch de capa 2 hacen una inundación de todo el tráfico multicast en todos los puertos de la misma VLAN.

Vídeo en un escenario IP

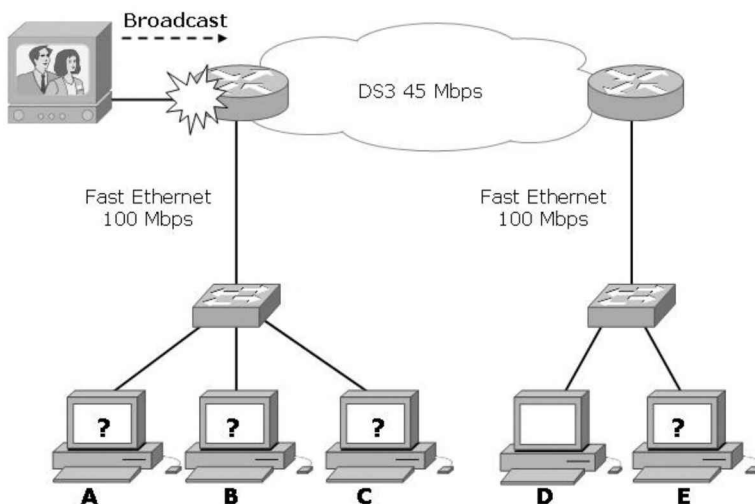
La siguiente figura muestra una topología en la que el jefe de un departamento de una empresa quiere hablar con los empleados utilizando vídeo sobre IP. Los enlaces Fast Ethernet son de 100 Mbps mientras que los enlaces WAN DS3 son de 45 Mbps. Los Usuarios A, B, C y E quieren atender el pedido pero el usuario D no.



Suponiendo que el flujo del vídeo fuera unicast y de 1 Mbps haría falta un camino de 1 Mbps hacia cada uno de los destinos. Los enlaces Fast Ethernet limitarían la cantidad de clientes a un máximo de 100, y los DS3 los limitarían a un máximo de 45 clientes por enlace. Si se utilizara todo el tráfico la red quedaría no disponible para el resto de los empleados.



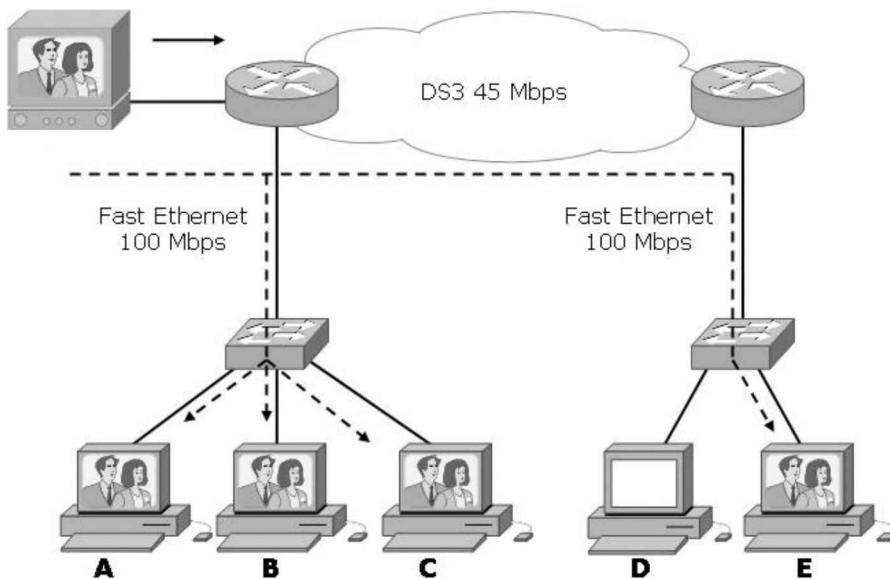
Cuando se reconoce este problema el administrador de la red trata de redistribuir el vídeo utilizando broadcast, pero no logra conexión debido a que los routers no procesan los broadcast, incluso dentro de una LAN los broadcast son problemáticos puesto que llegan a todos los usuarios obligándolos a procesar todo el tráfico.



El multicast resolvería esta situación. Un simple flujo multicast podría ser distribuido a todos los usuarios conservando la capacidad de la red.

El tráfico multicast se puede enrutar, y los usuarios que no desean verlo simplemente lo ignoran para no sobrecargar a los ordenadores.

En el peor escenario posible el usuario D estaría recibiendo el tráfico pero la tarjeta lo ignoraría. En el mejor caso posible el switch es lo suficientemente inteligente como para filtrar ese tráfico multicast sabiendo que D no quiere recibirlo. Para ello utilizaría **IGMP** o **CGMP**.



DIRECCIONAMIENTO MULTICAST

Los sistemas finales y tipos de los sistemas intermedios tienen que ser capaces de distinguir el tráfico multicast, unicast y broadcast.

Un dispositivo de capa 3 utiliza direcciones reservadas Clase D que van desde la 224.0.0.0 hasta la 239.255.255.255. Los dispositivos de red pueden reconocer rápidamente tráfico multicast porque los primeros 4 bits son siempre 1110. Los dispositivos Ethernet de manera similar tienen direcciones especiales para multicast.

El bit de menor orden en el primer byte de una dirección MAC sirve para indicar si es multicast o unicast, si el valor es 1 indica que la trama es multicast.

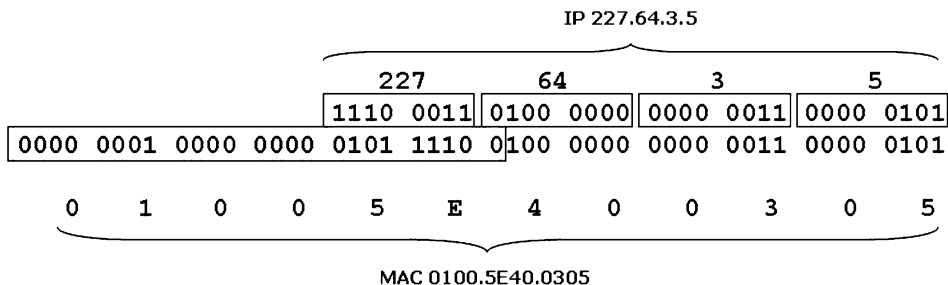
Además de este bit las direcciones IP multicast están mapeadas a un rango específico de direcciones MAC para ayudar a los host a discriminar el tipo de tráfico.

Direccionamiento MAC multicast

A nivel de capa 2 los dispositivos reconocen tráfico multicast porque cada dirección IP de multicast está asociada a una dirección MAC de 48 bits.

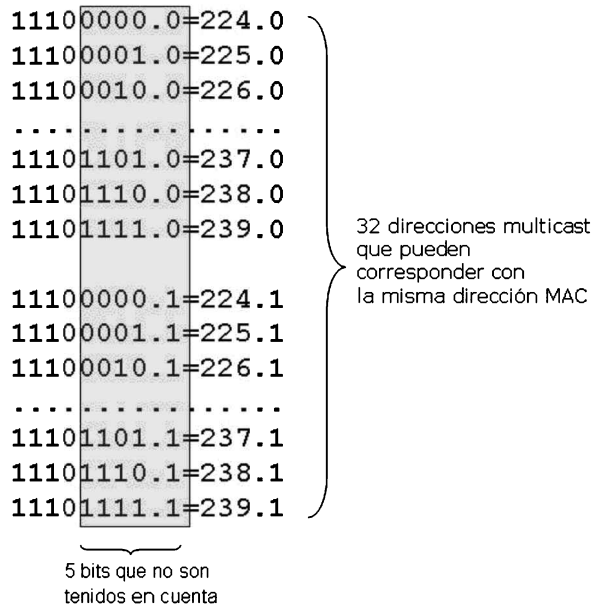
Los primeros 24 bits de la dirección MAC corresponden al **OUI** (Organizationally Unique Identifier) o código de fabricante y comienzan en multicast con 0100.5e y el bit número 25 siempre es 0. Los siguientes 23 bits de la dirección MAC son copiados desde los 23 bits más a la derecha de la dirección IP.

El siguiente ejemplo muestra el concepto del mapeo de direcciones:

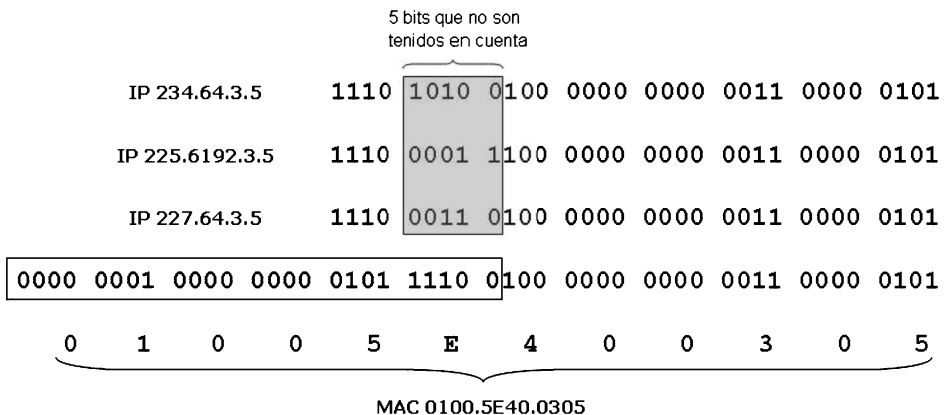


Solamente los primeros 23 bits desde la derecha son copiados desde la dirección IP a la MAC. Los prefijos más a la izquierda de la dirección IP y la dirección MAC son fijos y predecibles.

Hay 5 bits de la dirección IP dentro del rango que no son transferidos hacia de la dirección MAC multicast, esto crea la posibilidad de que las direcciones MAC de multicast no sean totalmente únicas. Existen 32 direcciones multicast diferentes que pueden corresponder con la misma dirección MAC.



Esta situación queda demostrada en la siguiente figura, donde la NIC es incapaz de distinguir tráfico para la dirección 234.64.3.5, 225.192.3.5 y 227.64.3.5.



Debido a esta ambigüedad el direccionamiento multicast encara un pequeño problema cuando está recibiendo una trama Ethernet que está destinada a una dirección MAC de multicast. Esta dirección MAC podría corresponder a 32 direcciones IP multicast diferentes. El host tiene que recibir y examinar cada trama que incluya la dirección MAC en la que está interesado, sin importar desde qué dirección IP está viajando dicha trama hacia él. El host examina la cabecera del

paquete para verificar que la dirección IP multicast específica corresponde al grupo deseado. Para el caso que no sea lo esperado la trama se descarta.

Direccionamiento IP multicast

Sumado al direccionamiento de Clase D de multicast existen varios rangos que se utilizan para un direccionamiento en particular:

Link-local addresses (224.0.0.0/24), son usadas en el segmento local, poseen un TTL de 1 de tal manera que los routers no reenvían el tráfico debido al valor del TTL. Estas direcciones son fijas y conocidas:

Dirección	Destino
224.0.0.1	Todos los hosts.
224.0.0.5	Todos los router OSPF.
224.0.0.6	Todos los DR OSPF.
224.0.0.9	Todos los router RIPv2.
224.0.0.10	Todos los router EIGRP.

Source-specific multicast (232.0.0.0/8), este direccionamiento es una extensión de multicast donde el host puede seleccionar el grupo del cual quiere recibir tráfico, o por el contrario lo recibe desde cualquier origen. Se utiliza con IGMPv3.

GLOP (233.0.0.0/8), este rango proporciona 256 direcciones IP de multicast para cada sistema autónomo que esté registrado. El número de AS de 16 bits es utilizado en los octetos centrales, en el ejemplo se observa un AS 1500:

233.0.0.0/8

AS 1500

Donde:

233 es el identificador GLOP

1500=10111011100 AS en binario

Por lo tanto **233.5.220.0**

Administratively scoped addresses (239.0.0.0/8), este espacio de direccionamiento es utilizado en dominios de multicast privados. Llevan una similitud con la RFC 1918. Estas direcciones no son enrutadas entre dominios de manera que pueden ser reutilizadas. Dentro de este ámbito administrativo la dirección 239.252.0.0/14 es utilizada localmente y la 239.192.0.0/10 para toda la organización.

Globally scoped addresses (224.0.1.0–231.255.255.255 y 234.0.0.0–238.255.255.255), este direccionamiento puede ser utilizado por cualquier entidad para enrutar tráfico para cualquier organización en Internet. Son únicas y globalmente significativas y son asignadas de manera temporal.

Aplicaciones multicast

El direccionamiento multicast es utilizado en muchas aplicaciones, como las que distribuyen imágenes a través de una red. En este escenario, un servidor es cargado con todas las imágenes y cada uno de los clientes puede recibirlas a través del tráfico multicast. Otro uso común de las aplicaciones multicast es un escenario compartido, donde varios usuarios comparten un espacio de trabajo.

Por último agregar que el direccionamiento multicast permite descubrir procesos de enrutamiento. Los procesos de enrutamiento de RIPv2, OSPF y EIGRP descubren a sus vecinos por medio del direccionamiento multicast.

Las aplicaciones pueden ser de uno a varios, como por ejemplo las imágenes de varios a varios, como compartir un escenario de trabajo o de varios a uno, como el caso de las actualizaciones de enrutamiento.

Problemas con multicast

Multicast presenta una serie de problemas debido a que el tráfico en UDP no existe una función de control de envío ni de organización de paquetes, ni la capacidad de retransmisión. Sumado a la falta de capacidad para permitir a ciertos usuarios en el mismo segmento recibir tráfico multicast y otros no, aunque existen métodos para corregir este comportamiento.

Estos retos pueden resolverse a nivel de protocolo. Para tráfico multimedia el orden puede ser controlado utilizando RTP (Real Time Protocol) y los problemas de seguridad pueden resolverse utilizando encriptación con IPsec.

Multicast y la capa de enlace

Los switch de capa 2 construyen una tabla que enlaza direcciones MAC con los puertos que llegan hacia ellas. Los switch aprenden acerca de cada dirección MAC y sus correspondientes puertos mirando la dirección MAC de origen cuando recibe tráfico. Cuando el switch se inicia desconoce ese mapeo al no poseer tablas almacenadas, por lo que inunda todos los puertos dentro del mismo dominio de broadcast o VLAN. Este mecanismo funciona bien para difusiones, una dirección de broadcast nunca será una dirección de origen por lo tanto nunca estará en la tabla de direcciones MAC. Los broadcast son siempre reenviados por todos los puertos.

El tráfico multicast también se lo identifica como desconocido porque las direcciones multicast solamente aparecen como dirección de destino. Los switch reenvían todo ese tráfico y es el destino final quien elegirá las tramas en que está interesado. El tráfico multicast hacia un dispositivo que no está interesado en él es un gasto de ancho de banda. Las tarjetas de red que no son suficientemente sofisticadas o modernas no son capaces de discriminar si están interesadas o no en las tramas que reciben, por lo tanto esto conlleva a que la decisión debe ser tomada por el procesador, gastando recursos del mismo.

Existen tres métodos en los switch Cisco para manejar y optimizar esta situación. Estos tres métodos lo que intentan es limitar el tráfico multicast a los puertos que realmente quieren recibirlo y ocultarlo a los demás:

- Entradas estáticas en las tablas MAC, mapeando manualmente las entradas MAC.
- CGMP (Cisco Group Management Protocol).
- IGMP (Internet Group Management Protocol).

Creando las entradas manualmente de forma estática es el método más sencillo de entender, pero no puede reaccionar ante cambios en la red, es decir, no es dinámico. Este sistema conlleva a la manipulación de muchos switch en la red generando una gran carga de trabajo. Crear las tablas MAC manualmente es más fácil pero no el método apropiado.

CGMP es una solución propietaria de Cisco. Es un protocolo que se ejecuta entre los switch y los routers. Cuando el router recibe mensajes IGMP pasa la MAC de quien ha solicitado ese tráfico multicast y el grupo multicast al cual se hace esa petición y reenvía la información al switch. El switch busca en su tabla MAC a quien ha efectuado dicha petición permitiendo que el tráfico multicast de ese grupo pase por el puerto determinado.

IGMP *snooping* es un método que permite al switch comprender cuál es el grupo que se está pidiendo y en qué puerto está el receptor. Este mecanismo se hace a nivel de procesador, pudiendo verse sobrecargado según las exigencias de la red. Los dispositivos Cisco actuales soportan IGMP incorporado en el propio hardware.

IGMP

Antes de que los routers puedan enviar tráfico multicast primero tienen que conocer la ubicación de los clientes. IGMP es el protocolo utilizado entre los clientes y el router para identificarse y solicitar el tráfico multicast al router. Para recibir tráfico multicast desde una fuente los destinos se unen a un grupo multicast común identificado por una dirección IP multicast. Un host se une a un grupo enviando una petición a su router local, ésta se lleva a cabo con el *Internet Group Management Protocol*.

Existen tres versiones de este protocolo IGMPv1, IGMPv2 e IGMPv3.

IGMPv1

Para unirse a un grupo de multicast un host envía un *Membership Report* IGMP, que es un mensaje que este protocolo utiliza para contactarse con el router local. Este mensaje indica al router la dirección o grupo al que quiere unirse ese host. La dirección de multicast es utilizada como la dirección IP de destino y la dirección de grupo es listada en el mensaje. Los routers multicast tienen que interceptar todos los paquetes multicast para recibir los *Membership Report* y añadir los host a los grupos apropiados.

Cada 60 segundos un router denominado *querier* en cada segmento de red envía una consulta a la dirección multicast de todos los host 224.0.0.1 para confirmar si hay alguno interesado en recibir un grupo multicast. Con que solamente un host responda mantendrá el puerto en un estado de reenvío. Los host que estén interesados responderán con *Membership Report* siempre y cuando nadie lo haya hecho con anterioridad.

Los host pueden unirse a un grupo multicast en cualquier momento, pero IGMPv1 no tiene un mecanismo que permita al host dejar el grupo cuando ya no está interesado en seguir recibiendo tráfico. Los routers manejan esta situación haciendo expirar la dirección de grupo multicast si nadie quiere recibir ese tráfico. Concretamente, si no se reciben reportes en 3 consultas consecutivas, el grupo expira. Esto significa que el tráfico multicast es enviado dentro del segmento hasta tres minutos después de que todos los miembros del grupo han dejado de escuchar.